Projet QueenBee : Algorithme de Dijkstra

Plus court chemin à source unique, interventions d'urgence (UC-C01)

Membres du groupe (binôme)

- Daniel Nagoloum TALLA
- Wissam AIT EL HADI

Spécialité : Développement / DevOps
Formation : Mastère 1 INGETIS / ITIS, application fil rouge QueenBee
Évaluation : Algorithmique (sujet B : Dijkstra, file de priorité / tas binaire)
Date de rendu : 05/11/2026 (Pronote), document généré le 02/11/2026
Dépôt : https://github.com/nagoloumdaniel/Algo-devoir

Compte rendu

1. Contexte et objectif

Le projet QueenBee déploie un réseau apicole connecté (8 sites, 3 pays, 400 ruches). Quand une ruche déclenche une alerte critique, le coordinateur régional doit la rejoindre au plus vite. Nous avons implémenté Dijkstra **sans aucune bibliothèque de graphes**, avec un **tas binaire (min-heap)** codé à la main, puis nous l'avons exposé dans le backend sous la forme de l'endpoint `POST /api/v1/routing/emergency-path`.

2. Membres du groupe et répartition

- **Daniel Nagoloum TALLA** : Tests E2E, captures, rapport PDF, dépôt GitHub
- **Wissam AIT EL HADI** : Algorithme, tas binaire, API, tests unitaires, benchmark

Tâches réalisées par Wissam AIT EL HADI

Tâches	Contenu	Fichiers
T-W1, T-W2	Dijkstra avec tas binaire (BinaryHeap codé de zéro : heapify, insert, extract_min, redimensionnement)	`src/min_heap.py, src/dijkstra.py`
T-W3, T-W4	Reconstitution du chemin optimal, coût cumulé, format « X min Y sec »	`src/dijkstra.py`
T-W5 à T-W7	Endpoint REST, schémas Pydantic d'entrée et de sortie, graphe du site, service de routage	`src/api.py, src/routing_service.py, src/site_graph.py`
T-W9, T-W10	Tests pytest : cas normaux et cas limites (55 tests)	`tests/`
T-W11 à T-W14	Version naïve O(|V|²), benchmark, note de complexité	`src/benchmark.py, docs/analyse_complexite.md`

Tâches réalisées par Daniel Nagoloum TALLA

Tâches	Contenu	Réalisation
T-D1 à T-D4	Architecture API : endpoint, modèles, validation des identifiants, routage URL	Relue et validée ; chaque comportement est vérifié de bout en bout (E2E, section 4).
T-D5 à T-D7	Intégration de Dijkstra, gestion des erreurs, format JSON	Vérifiées par les tests E2E : 200, 404, 422, 405, ruche isolée (status « error »).
T-D8 (API)	Documentation OpenAPI	Vérifiée dans Swagger UI par Playwright, captures nommées (section 4).
T-D8 à T-D11	Benchmarks, graphiques, note de complexité, analyse des résultats	Données, graphiques et note relus et intégrés au rapport (section 5).
T-D12, T-D13	Mise en page du PDF unique, taille < 5 Mo	Rapport généré par scripts/build_report.py (taille vérifiée à chaque génération).
T-D15	Traçabilité noms et prénoms	Page de garde avec les noms des deux membres du binôme.
T-D14	Dépôt sur Pronote	À faire par chaque étudiant avant le 05/11/2026 (action manuelle).

Daniel a de plus ajouté les **tests E2E Playwright** (8 tests, 7 captures d'écran) et corrigé le `.gitignore` (encodage cassé, fichiers `__pycache__` suivis par git).

3. Implémentation de l'algorithme (code source)

Code complet et commenté. Aucune bibliothèque de graphes n'est utilisée : seule la bibliothèque standard Python est employée pour l'algorithme ; FastAPI et Pydantic servent uniquement à la couche REST.

Structure de données : tas binaire (min-heap) `src/min_heap.py` (205 lignes)

```
"""
Tas binaire minimum (Min-Heap) implémenté de zéro, T-W2.

Aucune dépendance externe : le module ``heapq`` de Python n'est PAS utilisé.

Représentation
--------------
Le tas est un arbre binaire presque complet stocké dans un tableau :
    - la racine est à l'indice 0 ;
    - le parent de i est à (i - 1) // 2 ;
    - les enfants de i sont à 2i + 1 et 2i + 2.

Propriété de tas (min) : pour tout i > 0, tab[parent(i)] <= tab[i].
La racine contient donc toujours le plus petit élément.

Complexités
-----------
    insert       O(log n)   remontée (bubble_up) sur la hauteur de l'arbre
    extract_min  O(log n)   descente (bubble_down) sur la hauteur de l'arbre
    get_min      O(1)       lecture de la racine
    heapify      O(n)       construction ascendante (Floyd)
    size         O(1)

Le tableau interne a une capacité fixe qui double lorsqu'il est plein
(redimensionnement amorti en O(1) par insertion).

Exemple
-------
>>> h = BinaryHeap()
>>> h.insert(5); h.insert(1); h.insert(3)
>>> h.extract_min()
1
>>> h.get_min()
3
>>> len(h)
2
"""



class BinaryHeap:
    """Tas binaire minimum redimensionnable.

    Les éléments doivent être comparables entre eux avec ``<``.
    Pour Dijkstra on y range des tuples ``(distance, compteur, noeud)`` :
    la comparaison de tuples se fait sur la distance d'abord, et le compteur
    (unique) départage les égalités sans jamais avoir à comparer les noeuds.
    """

    DEFAULT_CAPACITY = 16

    def __init__(self, capacity=DEFAULT_CAPACITY):
        if capacity < 1:
            raise ValueError("La capacité initiale doit être >= 1")
        self._data = [None] * capacity   # tableau de taille fixe
        self._size = 0                   # nombre d'éléments réellement présents
        self.comparisons = 0             # compteur d'opérations (pour le benchmark)

    # ------------------------------------------------------------------ #
    # Accesseurs
    # ------------------------------------------------------------------ #
    def size(self):
```

```
        """Nombre d'éléments dans le tas. O(1)."""
        return self._size

    def __len__(self):
        return self._size

    def is_empty(self):
        """True si le tas ne contient aucun élément. O(1)."""
        return self._size == 0

    @property
    def capacity(self):
        """Capacité actuelle du tableau interne (avant prochain redimensionnement)."""
        return len(self._data)

    def get_min(self):
        """Renvoie le minimum SANS le retirer. O(1).

        >>> h = BinaryHeap(); h.insert(4); h.insert(2)
        >>> h.get_min()
        2
        """
        if self._size == 0:
            raise IndexError("get_min() sur un tas vide")
        return self._data[0]

    # ------------------------------------------------------------------ #
    # Opérations principales
    # ------------------------------------------------------------------ #
    def insert(self, item):
        """Ajoute un élément. O(log n) (O(1) amorti pour le redimensionnement).

        L'élément est placé dans la première case libre (dernière feuille),
        puis remonté tant qu'il est plus petit que son parent.
        """
        if self._size == len(self._data):
            self._resize(2 * len(self._data))
        self._data[self._size] = item
        self._size += 1
        self._bubble_up(self._size - 1)

    def extract_min(self):
        """Retire et renvoie le plus petit élément. O(log n).

        La racine est remplacée par la dernière feuille, qui est ensuite
        redescendue jusqu'à ce que la propriété de tas soit rétablie.
        """
        if self._size == 0:
            raise IndexError("extract_min() sur un tas vide")
        minimum = self._data[0]
        self._size -= 1
        last = self._data[self._size]
        self._data[self._size] = None        # libère la référence
        if self._size > 0:
            self._data[0] = last
            self._bubble_down(0)
        return minimum

    @classmethod
    def heapify(cls, items):
        """Construit un tas à partir d'un itérable en O(n) (algorithme de Floyd).

        On recopie les éléments tels quels, puis on fait descendre chaque
        noeud interne, du dernier vers la racine. La somme des hauteurs des
        noeuds d'un arbre presque complet est O(n), d'où le coût linéaire
        (contre O(n log n) pour n insertions successives).

        >>> BinaryHeap.heapify([5, 3, 8, 1]).extract_min()
        1
        """
        items = list(items)
        heap = cls(capacity=max(cls.DEFAULT_CAPACITY, len(items)))
        for i, item in enumerate(items):
            heap._data[i] = item
        heap._size = len(items)
        # Le dernier noeud interne est le parent de la dernière feuille.
        for i in range(heap._size // 2 - 1, -1, -1):
            heap._bubble_down(i)
        return heap

    # ------------------------------------------------------------------ #
```

```
    # Mécanique interne
    # ------------------------------------------------------------------ #
    def _less(self, i, j):
        """Compare data[i] < data[j] en comptant la comparaison."""
        self.comparisons += 1
        return self._data[i] < self._data[j]

    def _swap(self, i, j):
        self._data[i], self._data[j] = self._data[j], self._data[i]

    def _bubble_up(self, index):
        """Remonte l'élément en ``index`` tant qu'il est < à son parent."""
        while index > 0:
            parent = (index - 1) // 2
            if self._less(index, parent):
                self._swap(index, parent)
                index = parent
            else:
                break

    def _bubble_down(self, index):
        """Descend l'élément en ``index`` vers son plus petit enfant si besoin."""
        while True:
            left = 2 * index + 1
            right = left + 1
            smallest = index
            if left < self._size and self._less(left, smallest):
                smallest = left
            if right < self._size and self._less(right, smallest):
                smallest = right
            if smallest == index:
                break
            self._swap(index, smallest)
            index = smallest

    def _resize(self, new_capacity):
        """Recopie les éléments dans un tableau de ``new_capacity`` cases. O(n)."""
        new_data = [None] * new_capacity
        for i in range(self._size):
            new_data[i] = self._data[i]
        self._data = new_data

    # ------------------------------------------------------------------ #
    # Vérification et affichage
    # ------------------------------------------------------------------ #
    def is_valid(self):
        """Vérifie la propriété de tas sur tout le tableau (utile en test)."""
        for i in range(1, self._size):
            if self._data[i] < self._data[(i - 1) // 2]:
                return False
        return True

    def to_list(self):
        """Copie des éléments dans l'ordre du tableau interne (pas trié)."""
        return self._data[:self._size]

    def __str__(self):
        return f"BinaryHeap({self.to_list()})"

    def __repr__(self):
        return (f"BinaryHeap(size={self._size}, capacity={len(self._data)}, "
                f"data={self.to_list()})")
```

Dijkstra avec tas, version naïve, reconstitution du chemin `src/dijkstra.py` (336 lignes)

```
"""
Algorithme de Dijkstra, projet QueenBee, cas d'usage UC-C01 (interventions d'urgence).

Contenu (tâches de Wissam) :
    T-W1  dijkstra()            Dijkstra avec file de priorité (tas binaire maison)
    T-W3  reconstruct_path()    reconstitution du chemin optimal source -> cible
    T-W4  shortest_path()       distance / temps cumulé + nombre d'étapes
          format_duration()     formatage "X min Y sec"
    T-W12 dijkstra_naive()      version naïve O(|V|²) sans tas (recherche linéaire)

Aucune bibliothèque de graphes (NetworkX, SciPy...) n'est utilisée.

Format du graphe
```

```
----------------
Liste d'adjacence sous forme de dictionnaire. Deux variantes sont acceptées :
    {"A": {"B": 4.0, "C": 2.0}, ...}          dict de dict
    {"A": [("B", 4.0), ("C", 2.0)], ...}      dict de listes de couples
Une liste d'arêtes peut être convertie avec build_graph().
Les poids doivent être >= 0 (condition de validité de Dijkstra).
"""

import argparse
import math

from src.min_heap import BinaryHeap

INFINITY = math.inf


# ================================================================== #
# Construction / normalisation du graphe
# ================================================================== #
def build_graph(edges, directed=False):
    """Construit une liste d'adjacence {u: {v: poids}} à partir d'arêtes (u, v, poids).

    Si ``directed`` est False, chaque arête est ajoutée dans les deux sens
    (une piste se parcourt dans les deux sens). En cas d'arêtes multiples
    entre deux mêmes noeuds, on garde la plus courte.

    >>> build_graph([("A", "B", 3)])
    {'A': {'B': 3.0}, 'B': {'A': 3.0}}
    """
    graph = {}

    def add(u, v, w):
        graph.setdefault(u, {})
        graph.setdefault(v, {})
        if v not in graph[u] or w < graph[u][v]:
            graph[u][v] = w

    for u, v, weight in edges:
        weight = float(weight)
        if weight < 0:
            raise ValueError(f"Poids négatif interdit sur l'arête {u}-{v} : {weight}")
        add(u, v, weight)
        if not directed:
            add(v, u, weight)
    return graph


def normalize_graph(graph):
    """Renvoie une copie du graphe au format {u: {v: poids}} et la valide.

    - accepte les voisins sous forme de dict ou de liste de couples ;
    - ajoute comme noeuds isolés les sommets qui n'apparaissent qu'en destination ;
    - rejette les poids négatifs (Dijkstra serait faux).
    """
    if graph is None:
        raise ValueError("Le graphe ne peut pas être None")
    normalized = {}
    for u, neighbors in graph.items():
        items = neighbors.items() if isinstance(neighbors, dict) else neighbors
        normalized.setdefault(u, {})
        for v, weight in items:
            weight = float(weight)
            if weight < 0:
                raise ValueError(f"Poids négatif interdit sur l'arête {u}->{v} : {weight}")
            normalized.setdefault(v, {})
            if v not in normalized[u] or weight < normalized[u][v]:
                normalized[u][v] = weight
    return normalized


def _check_node(graph, node, role):
    if node not in graph:
        raise ValueError(f"Le noeud {role} '{node}' n'existe pas dans le graphe")


# ================================================================== #
# T-W1 : Dijkstra avec tas binaire : O((|V| + |E|) log |V|)
# ================================================================== #
def dijkstra(graph, source, target=None, stats=None, normalize=True):
    """Plus courts chemins depuis ``source`` avec une file de priorité (Min-Heap).

```

```
    Paramètres
    ----------
    graph  : liste d'adjacence (voir en-tête du module)
    source : noeud de départ
    target : optionnel ; si fourni, on s'arrête dès que la cible est extraite
             du tas (sa distance est alors définitive)
    stats  : dict optionnel rempli avec des compteurs d'opérations
    normalize : si False, le graphe est supposé déjà au format {u: {v: poids>=0}}
             (évite la copie O(|V|+|E|) ; utilisé par le benchmark pour ne
             chronométrer que l'algorithme)

    Renvoie
    -------
    (distances, predecessors)
        distances[v]    : coût minimal source -> v (math.inf si inaccessible)
        predecessors[v] : noeud précédent sur le meilleur chemin (None pour la
                          source et les noeuds inaccessibles)

    Principe
    --------
    Le tas contient des couples (distance provisoire, noeud). À chaque tour on
    extrait le noeud de plus petite distance : comme les poids sont >= 0, cette
    distance ne peut plus diminuer, le noeud est donc "définitif" (visited).
    On relâche ensuite ses arêtes sortantes.

    Plutôt qu'une opération decrease-key, on utilise la "suppression paresseuse" :
    quand une distance s'améliore on ré-insère le noeud ; les anciennes entrées
    périmées sont ignorées à l'extraction. Le tas contient au plus |E| + 1
    entrées, donc chaque opération coûte O(log |E|) = O(log |V|).
    """
    if normalize:
        graph = normalize_graph(graph)
    _check_node(graph, source, "source")
    if target is not None:
        _check_node(graph, target, "cible")

    distances = {node: INFINITY for node in graph}
    predecessors = {node: None for node in graph}
    distances[source] = 0.0

    visited = set()
    heap = BinaryHeap()
    counter = 0                       # départage les égalités de distance
    heap.insert((0.0, counter, source))
    pushes, pops, relaxations = 1, 0, 0

    while not heap.is_empty():
        current_dist, _, node = heap.extract_min()
        pops += 1

        if node in visited:           # entrée périmée (suppression paresseuse)
            continue
        visited.add(node)

        if node == target:            # arrêt anticipé : distance de la cible définitive
            break

        for neighbor, weight in graph[node].items():
            if neighbor in visited:
                continue
            relaxations += 1
            new_dist = current_dist + weight
            if new_dist < distances[neighbor]:       # relâchement
                distances[neighbor] = new_dist
                predecessors[neighbor] = node
                counter += 1
                heap.insert((new_dist, counter, neighbor))
                pushes += 1

    if stats is not None:
        stats.update({
            "heap_pushes": pushes,
            "heap_pops": pops,
            "relaxations": relaxations,
            "comparisons": heap.comparisons,
            "visited": len(visited),
        })
    return distances, predecessors


# ============================================================================ #
```

```python
# T-W12 : version naïve sans tas : O(|V|²)
# ============================================================================ #
def dijkstra_naive(graph, source, target=None, stats=None, normalize=True):
    """Même résultat que dijkstra(), mais le minimum est cherché par parcours linéaire.

    Chacune des |V| itérations parcourt tous les noeuds non visités pour trouver
    le plus proche : O(|V|) par itération, donc O(|V|²) au total, plus O(|E|)
    pour les relâchements. Aucune file de priorité.
    """
    if normalize:
        graph = normalize_graph(graph)
    _check_node(graph, source, "source")
    if target is not None:
        _check_node(graph, target, "cible")

    distances = {node: INFINITY for node in graph}
    predecessors = {node: None for node in graph}
    distances[source] = 0.0
    unvisited = set(graph)
    comparisons, relaxations = 0, 0

    while unvisited:
        # Recherche linéaire du noeud non visité de distance minimale.
        node, best = None, INFINITY
        for candidate in unvisited:
            comparisons += 1
            if distances[candidate] < best:
                node, best = candidate, distances[candidate]
        if node is None:             # tous les restants sont inaccessibles
            break
        unvisited.remove(node)
        if node == target:
            break
        for neighbor, weight in graph[node].items():
            if neighbor not in unvisited:
                continue
            relaxations += 1
            new_dist = best + weight
            if new_dist < distances[neighbor]:
                distances[neighbor] = new_dist
                predecessors[neighbor] = node

    if stats is not None:
        stats.update({
            "comparisons": comparisons,
            "relaxations": relaxations,
            "visited": len(graph) - len(unvisited),
        })
    return distances, predecessors


# ============================================================================ #
# T-W3 : reconstitution du chemin optimal
# ============================================================================ #
def reconstruct_path(predecessors, source, target, distances=None):
    """Renvoie la liste ordonnée [source, ..., target], ou None s'il n'y a pas de chemin.

    On part de la cible et on remonte les prédécesseurs jusqu'à la source
    (predecessors[source] vaut None), puis on inverse la liste. O(longueur du chemin).
    """
    if source not in predecessors:
        raise ValueError(f"Le noeud source '{source}' n'existe pas dans le graphe")
    if target not in predecessors:
        raise ValueError(f"Le noeud cible '{target}' n'existe pas dans le graphe")
    if source == target:
        return [source]
    if distances is not None and distances.get(target, INFINITY) == INFINITY:
        return None

    path = []
    node = target
    while node is not None:
        path.append(node)
        if node == source:
            path.reverse()
            return path
        node = predecessors[node]
        if len(path) > len(predecessors):    # garde-fou contre une table corrompue
            raise RuntimeError("Cycle détecté dans la table des prédécesseurs")
    return None                              # la remontée n'a pas atteint la source

```

```python

# ============================================================================ #
# T-W4 : distance / temps cumulé
# ============================================================================ #
def path_cost(graph, path):
    """Recalcule le coût d'un chemin en additionnant le poids de ses arêtes.

    Sert de vérification croisée avec distances[cible].
    """
    graph = normalize_graph(graph)
    total = 0.0
    for u, v in zip(path, path[1:]):
        if v not in graph[u]:
            raise ValueError(f"L'arête {u}->{v} n'existe pas")
        total += graph[u][v]
    return total


def format_duration(minutes):
    """Formate une durée exprimée en minutes en "X min Y sec".

    >>> format_duration(12.5)
    '12 min 30 sec'
    >>> format_duration(0)
    '0 min 0 sec'
    """
    if minutes is None or minutes == INFINITY:
        return "inaccessible"
    total_seconds = round(minutes * 60)
    return f"{total_seconds // 60} min {total_seconds % 60} sec"


def shortest_path(graph, source, target, use_heap=True):
    """Calcule le meilleur itinéraire source -> cible et le coût associé.

    Renvoie un dictionnaire :
        reachable  : bool
        path       : liste ordonnée des noeuds, ou None si inaccessible
        total_cost : coût total arrondi à 2 décimales, ou None si inaccessible
        hop_count  : nombre d'arêtes parcourues, ou None si inaccessible
    """
    algorithm = dijkstra if use_heap else dijkstra_naive
    distances, predecessors = algorithm(graph, source, target)
    path = reconstruct_path(predecessors, source, target, distances)
    if path is None:
        return {"reachable": False, "path": None, "total_cost": None, "hop_count": None}
    return {
        "reachable": True,
        "path": path,
        "total_cost": round(distances[target], 2),
        "hop_count": len(path) - 1,
    }


# ============================================================================ #
# Point d'entrée en ligne de commande :  python -m src.dijkstra --help
# ============================================================================ #
def main(argv=None):
    from src.site_graph import SITE_GRAPH

    parser = argparse.ArgumentParser(
        description="Plus court chemin (Dijkstra + tas binaire) sur le site de démo QueenBee.")
    parser.add_argument("--source", default="BASE_NORD", help="noeud de départ")
    parser.add_argument("--target", default="H07", help="noeud d'arrivée (ruche)")
    parser.add_argument("--naive", action="store_true", help="utiliser la version O(|V|²)")
    args = parser.parse_args(argv)

    result = shortest_path(SITE_GRAPH, args.source, args.target, use_heap=not args.naive)
    if not result["reachable"]:
        print(f"Aucun chemin entre {args.source} et {args.target}.")
        return 1
    print("Chemin   :", " -> ".join(result["path"]))
    print("Durée    :", format_duration(result["total_cost"]),
          f"({result['total_cost']} min)")
    print("Étapes   :", result["hop_count"])
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Graphe de démonstration du site (pistes et accès) `src/site_graph.py` (58 lignes)

```python
"""
Graphe de démonstration d'un site QueenBee : maillage des pistes et accès.

Hypothèse de modélisation (à remplacer par les données réelles du CDC si besoin) :
    - sommets  : bases des coordinateurs (BASE_*), carrefours de pistes (J*), ruches (H**)
    - arêtes   : tronçons de piste praticables, non orientés
    - poids    : temps de trajet en MINUTES (moto / pick-up)

H13 est volontairement isolée (piste coupée) pour illustrer le cas "aucun chemin".
Le graphe contient des cycles (ex. BASE_NORD - J1 - J2 - BASE_NORD).
"""

from src.dijkstra import build_graph

# (noeud A, noeud B, temps en minutes)
SITE_EDGES = [
    # Bases des coordinateurs vers le réseau de pistes
    ("BASE_NORD", "J1", 4.0),
    ("BASE_NORD", "J2", 9.0),
    ("BASE_SUD", "J5", 3.5),
    ("BASE_SUD", "J6", 6.0),
    # Pistes principales entre carrefours
    ("J1", "J2", 3.0),
    ("J1", "J3", 7.5),
    ("J2", "J3", 2.5),
    ("J2", "J4", 8.0),
    ("J3", "J4", 4.0),
    ("J3", "J5", 11.0),
    ("J4", "J6", 5.0),
    ("J5", "J6", 4.5),
    # Accès aux ruches
    ("J1", "H01", 2.0),
    ("J1", "H02", 3.5),
    ("J2", "H03", 1.5),
    ("J3", "H04", 2.0),
    ("J3", "H05", 3.0),
    ("H04", "H05", 1.5),
    ("J4", "H06", 2.5),
    ("J4", "H07", 4.0),
    ("H06", "H07", 2.0),
    ("J5", "H08", 2.0),
    ("J5", "H09", 3.0),
    ("J6", "H10", 1.5),
    ("J6", "H11", 2.5),
    ("H10", "H12", 3.0),
]

SITE_GRAPH = build_graph(SITE_EDGES)
SITE_GRAPH["H13"] = {}           # ruche isolée : accès coupé

# Coordinateurs régionaux -> point de départ dans le graphe
COORDINATORS = {
    "C01": "BASE_NORD",
    "C02": "BASE_SUD",
}

# Ruches connues du site
HIVES = sorted(node for node in SITE_GRAPH if node.startswith("H"))
```

Service de routage d'urgence `src/routing_service.py` (94 lignes)

```python
"""
Service de calcul d'itinéraire d'urgence (UC-C01).

Couche métier appelée par l'endpoint REST (src/api.py). Elle ne dépend pas de
FastAPI, ce qui permet de la tester directement et de garder l'API "fine".

Entrée : identifiant du coordinateur + identifiant de la ruche à risque.
Sortie : dictionnaire JSON-sérialisable (format de sortie de T-W7).
"""

import re
import time

from src.dijkstra import dijkstra, format_duration, reconstruct_path
```

```python
from src.site_graph import COORDINATORS, HIVES, SITE_GRAPH

ALGORITHM_VERSION = "dijkstra-minheap-1.0"

COORDINATOR_ID_PATTERN = re.compile(r"^C\d{2}$")   # ex. C01
HIVE_ID_PATTERN = re.compile(r"^H\d{2}$")          # ex. H07


class InvalidIdentifierError(ValueError):
    """Identifiant mal formé  -> HTTP 400."""


class UnknownResourceError(LookupError):
    """Coordinateur ou ruche inexistant  -> HTTP 404."""


def normalize_id(value, field_name):
    """Supprime les espaces et met en majuscules ("h07 " -> "H07")."""
    if not isinstance(value, str):
        raise InvalidIdentifierError(f"{field_name} doit être une chaîne de caractères")
    return value.strip().upper()


def compute_emergency_path(coordinator_id, target_hive_id,
                           graph=SITE_GRAPH, coordinators=COORDINATORS, hives=HIVES):
    """Calcule l'itinéraire optimal du coordinateur vers la ruche en alerte.

    Lève InvalidIdentifierError (400) ou UnknownResourceError (404).
    Si la ruche existe mais n'est pas accessible, renvoie status="error"
    avec les valeurs par défaut (path vide, total_cost None).
    """
    coordinator_id = normalize_id(coordinator_id, "coordinator_id")
    target_hive_id = normalize_id(target_hive_id, "target_hive_id")

    if not COORDINATOR_ID_PATTERN.match(coordinator_id):
        raise InvalidIdentifierError(
            f"coordinator_id '{coordinator_id}' invalide (format attendu : CXX, ex. C01)")
    if not HIVE_ID_PATTERN.match(target_hive_id):
        raise InvalidIdentifierError(
            f"target_hive_id '{target_hive_id}' invalide (format attendu : HXX, ex. H07)")
    if coordinator_id not in coordinators:
        raise UnknownResourceError(f"Coordinateur '{coordinator_id}' inconnu")
    if target_hive_id not in hives or target_hive_id not in graph:
        raise UnknownResourceError(f"Ruche '{target_hive_id}' inconnue")

    start_node = coordinators[coordinator_id]

    # Seul l'algorithme est chronométré (pas la validation ni la sérialisation).
    t0 = time.perf_counter()
    distances, predecessors = dijkstra(graph, start_node, target_hive_id)
    path = reconstruct_path(predecessors, start_node, target_hive_id, distances)
    elapsed_ms = (time.perf_counter() - t0) * 1000

    response = {
        "coordinator_id": coordinator_id,
        "target_hive_id": target_hive_id,
        "start_node": start_node,
        "calculation_time_ms": round(elapsed_ms, 4),
        "algorithm_version": ALGORITHM_VERSION,
    }
    if path is None:
        response.update({
            "status": "error",
            "message": f"Aucun itinéraire praticable vers la ruche {target_hive_id}",
            "total_cost": None,
            "total_cost_formatted": None,
            "path": [],
            "hop_count": 0,
        })
    else:
        total = round(distances[target_hive_id], 2)
        response.update({
            "status": "success",
            "message": None,
            "total_cost": total,
            "total_cost_formatted": format_duration(total),
            "path": path,
            "hop_count": len(path) - 1,
        })
    return response
```

API REST FastAPI `src/api.py` (148 lignes)

```python
"""
API REST QueenBee : routage d'urgence (T-W5, T-W6, T-W7).

    POST /api/v1/routing/emergency-path

Lancement :
    uvicorn src.api:app --reload
    Documentation interactive : http://127.0.0.1:8000/docs
"""

import logging
from enum import Enum
from typing import Any, Dict, List, Optional

from fastapi import APIRouter, FastAPI, HTTPException
from pydantic import BaseModel, Field, field_validator

from src.routing_service import (
    InvalidIdentifierError,
    UnknownResourceError,
    compute_emergency_path,
)

logger = logging.getLogger("queenbee.routing")
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(name)s: %(message)s")


# ============================================================================ #
# T-W6 : schéma d'entrée
# ============================================================================ #
class UrgencyLevel(str, Enum):
    critique = "critique"
    haute = "haute"
    moyenne = "moyenne"


class EmergencyPathRequest(BaseModel):
    coordinator_id: str = Field(
        ..., pattern=r"^C\d{2}$",
        description="Identifiant du coordinateur régional (point de départ), format CXX.",
        examples=["C01"],
    )
    target_hive_id: str = Field(
        ..., pattern=r"^H\d{2}$",
        description="Identifiant de la ruche en alerte critique, format HXX.",
        examples=["H07"],
    )
    urgence_niveau: Optional[UrgencyLevel] = Field(
        None, description="Niveau d'urgence de l'alerte (optionnel).")
    metadata: Optional[Dict[str, Any]] = Field(
        None, description="Informations libres sur l'alerte (optionnel).",
        examples=[{"alerte": "chute_de_poids"}])

    @field_validator("coordinator_id", "target_hive_id", mode="before")
    @classmethod
    def normalize(cls, value):
        """Insensible à la casse et aux espaces : ' h07' -> 'H07'.
        Les valeurs non textuelles sont laissées telles quelles et rejetées par Pydantic."""
        if isinstance(value, str):
            return value.strip().upper()
        return value

    model_config = {
        "json_schema_extra": {
            "examples": [{"coordinator_id": "C01", "target_hive_id": "H07",
                          "urgence_niveau": "critique"}]
        }
    }


# ============================================================================ #
# T-W7 : schéma de sortie
# ============================================================================ #
class EmergencyPathResponse(BaseModel):
    status: str = Field(..., description='"success" ou "error" (ruche inaccessible).')
    message: Optional[str] = Field(None, description="Détail en cas d'erreur.")
    coordinator_id: str
```

```python
    target_hive_id: str
    start_node: str = Field(..., description="Noeud de départ du coordinateur dans le graphe.")
    total_cost: Optional[float] = Field(
        None, description="Temps de trajet total en minutes (2 décimales), null si inaccessible.")
    total_cost_formatted: Optional[str] = Field(None, description='Ex. "12 min 30 sec".')
    path: List[str] = Field(..., description="Points de passage ordonnés, de la base à la ruche.")
    hop_count: int = Field(..., description="Nombre de tronçons parcourus.")
    calculation_time_ms: float = Field(..., description="Temps de calcul de l'algorithme (ms).")
    algorithm_version: str

    model_config = {
        "json_schema_extra": {
            "examples": [{
                "status": "success", "message": None,
                "coordinator_id": "C01", "target_hive_id": "H07", "start_node": "BASE_NORD",
                "total_cost": 17.5, "total_cost_formatted": "17 min 30 sec",
                "path": ["BASE_NORD", "J1", "J2", "J3", "J4", "H07"], "hop_count": 5,
                "calculation_time_ms": 0.042, "algorithm_version": "dijkstra-minheap-1.0",
            }]
        }
    }


class ErrorResponse(BaseModel):
    detail: str


# ========================================================================== #
# T-W5 : endpoint
# ========================================================================== #
router = APIRouter(prefix="/api/v1/routing", tags=["routing"])


@router.post(
    "/emergency-path",
    response_model=EmergencyPathResponse,
    summary="Itinéraire d'urgence vers une ruche en alerte (Dijkstra)",
    responses={400: {"model": ErrorResponse}, 404: {"model": ErrorResponse}},
)
def emergency_path(request: EmergencyPathRequest) -> EmergencyPathResponse:
    """Calcule le plus court chemin entre la base du coordinateur et la ruche cible.

    - **200** : calcul effectué (`status` = `success`, ou `error` si la ruche est inaccessible)
    - **400** : identifiant invalide
    - **404** : coordinateur ou ruche inconnu
    - **422** : corps de requête invalide (champ manquant, mauvais type, mauvais format)
    """
    logger.info("emergency-path coordinator=%s hive=%s urgence=%s",
                request.coordinator_id, request.target_hive_id, request.urgence_niveau)
    try:
        result = compute_emergency_path(request.coordinator_id, request.target_hive_id)
    except InvalidIdentifierError as exc:
        raise HTTPException(status_code=400, detail=str(exc))
    except UnknownResourceError as exc:
        raise HTTPException(status_code=404, detail=str(exc))
    logger.info("-> status=%s cost=%s time=%sms", result["status"],
                result["total_cost"], result["calculation_time_ms"])
    return EmergencyPathResponse(**result)


app = FastAPI(
    title="QueenBee API",
    version="1.0.0",
    description="Backend QueenBee : calcul d'itinéraires d'urgence (UC-C01).",
)
app.include_router(router)


if __name__ == "__main__":
    import uvicorn
    uvicorn.run("src.api:app", host="127.0.0.1", port=8000, reload=True)
```

4. Jeux de tests et résultats

4.1 Tests unitaires (pytest)

Résultat : **59 passed in 0.66s**. Cas normaux, cas limites (graphe non connexe, cycles, source = cible, graphe vide, graphe à un seul nœud), reconstitution du chemin, coût total, cohérence tas / version naïve, tas binaire, service et API.

```
============================= test session starts ==============================
collecting ... collected 59 items
tests/test_api.py::test_post_valide_renvoie_200_et_le_chemin PASSED      [  1%]
tests/test_api.py::test_champs_optionnels_acceptes PASSED                [  3%]
tests/test_api.py::test_ids_en_minuscules_acceptes PASSED                [  5%]
tests/test_api.py::test_champ_manquant_renvoie_422 PASSED                [  6%]
tests/test_api.py::test_mauvais_type_renvoie_422 PASSED                  [  8%]
tests/test_api.py::test_mauvais_format_renvoie_422 PASSED                [ 10%]
tests/test_api.py::test_niveau_urgence_inconnu_renvoie_422 PASSED        [ 11%]
tests/test_api.py::test_ruche_inexistante_renvoie_404 PASSED             [ 13%]
tests/test_api.py::test_coordinateur_inexistant_renvoie_404 PASSED       [ 15%]
tests/test_api.py::test_ruche_inaccessible_renvoie_status_error PASSED   [ 16%]
tests/test_api.py::test_get_renvoie_405 PASSED                           [ 18%]
tests/test_api.py::test_documentation_openapi_disponible PASSED          [ 20%]
tests/test_dijkstra.py::test_distances_connues PASSED                    [ 22%]
tests/test_dijkstra.py::test_chemin_connu PASSED                         [ 23%]
tests/test_dijkstra.py::test_cout_du_chemin_egal_a_la_distance PASSED    [ 25%]
tests/test_dijkstra.py::test_chemin_commence_a_la_source_et_finit_a_la_cible PASSED [ 27%]
tests/test_dijkstra.py::test_symetrie_graphe_non_oriente PASSED          [ 28%]
tests/test_dijkstra.py::test_inegalite_triangulaire PASSED               [ 30%]
tests/test_dijkstra.py::test_graphe_oriente PASSED                       [ 32%]
tests/test_dijkstra.py::test_format_liste_de_couples PASSED              [ 33%]
tests/test_dijkstra.py::test_arete_multiple_garde_la_plus_courte PASSED  [ 35%]
tests/test_dijkstra.py::test_poids_flottants_precision PASSED            [ 37%]
tests/test_dijkstra.py::test_arret_anticipe_donne_la_meme_distance PASSED [ 38%]
tests/test_dijkstra.py::test_format_duration PASSED                      [ 40%]
tests/test_dijkstra.py::test_graphe_non_connexe PASSED                   [ 42%]
tests/test_dijkstra.py::test_cycles_terminent PASSED                     [ 44%]
tests/test_dijkstra.py::test_boucle_sur_soi_meme PASSED                  [ 45%]
tests/test_dijkstra.py::test_source_egale_cible PASSED                   [ 47%]
tests/test_dijkstra.py::test_graphe_vide_leve_une_erreur_propre PASSED   [ 49%]
tests/test_dijkstra.py::test_graphe_a_un_seul_noeud PASSED               [ 50%]
tests/test_dijkstra.py::test_noeud_isole PASSED                          [ 52%]
tests/test_dijkstra.py::test_noeud_present_seulement_en_destination PASSED [ 54%]
tests/test_dijkstra.py::test_source_inexistante PASSED                   [ 55%]
tests/test_dijkstra.py::test_cible_inexistante PASSED                    [ 57%]
tests/test_dijkstra.py::test_poids_negatif_refuse PASSED                 [ 59%]
tests/test_dijkstra.py::test_naif_sur_le_graphe_exemple PASSED           [ 61%]
tests/test_dijkstra.py::test_tas_et_naif_identiques_sur_tout_le_site PASSED [ 62%]
tests/test_dijkstra.py::test_naif_graphe_non_connexe PASSED              [ 64%]
tests/test_dijkstra.py::test_naif_source_inexistante PASSED              [ 66%]
tests/test_min_heap.py::test_heapify_respecte_la_propriete_de_tas PASSED [ 67%]
tests/test_min_heap.py::test_heapify_liste_vide PASSED                   [ 69%]
tests/test_min_heap.py::test_insert_met_a_jour_taille_et_propriete PASSED [ 71%]
tests/test_min_heap.py::test_extract_min_renvoie_les_elements_dans_l_ordre PASSED [ 72%]
tests/test_min_heap.py::test_get_min_ne_modifie_pas_le_tas PASSED        [ 74%]
tests/test_min_heap.py::test_redimensionnement_conserve_les_elements PASSED [ 76%]
tests/test_min_heap.py::test_doublons_et_tuples PASSED                   [ 77%]
tests/test_min_heap.py::test_operations_sur_tas_vide_levent_une_erreur PASSED [ 79%]
tests/test_min_heap.py::test_capacite_invalide PASSED                    [ 81%]
tests/test_min_heap.py::test_str_et_repr PASSED                          [ 83%]
tests/test_routing_service.py::test_itineraire_connu PASSED              [ 84%]
tests/test_routing_service.py::test_structure_json_complete_et_serialisable PASSED [ 86%]
tests/test_routing_service.py::test_ids_insensibles_a_la_casse_et_aux_espaces PASSED [ 88%]
tests/test_routing_service.py::test_ruche_inaccessible_renvoie_les_valeurs_par_defaut PASSED [ 89%]
tests/test_routing_service.py::test_format_invalide[X01-H01] PASSED      [ 91%]
tests/test_routing_service.py::test_format_invalide[C01-R01] PASSED      [ 93%]
tests/test_routing_service.py::test_format_invalide[C1-H01] PASSED       [ 94%]
tests/test_routing_service.py::test_type_invalide PASSED                 [ 96%]
tests/test_routing_service.py::test_ruche_inexistante PASSED             [ 98%]
tests/test_routing_service.py::test_coordinateur_inexistant PASSED       [100%]
============================== 59 passed in 0.66s ==============================
```

Tests de Dijkstra `tests/test_dijkstra.py` (210 lignes)

```
"""Tests unitaires de Dijkstra : cas normaux (T-W9), cas limites (T-W10),
chemin (T-W3) et coût (T-W4)."""
```

```python

import math

import pytest

from src.dijkstra import (
    build_graph,
    dijkstra,
    dijkstra_naive,
    format_duration,
    path_cost,
    reconstruct_path,
    shortest_path,
)
from src.site_graph import SITE_GRAPH


@pytest.fixture
def sample_graph():
    """Graphe non orienté de 6 noeuds dont les plus courts chemins depuis A sont connus :
    A=0, C=2, B=3 (A-C-B), D=8 (A-C-B-D), E=10 (…-D-E), F=12 (…-E-F)."""
    return build_graph([
        ("A", "B", 4), ("A", "C", 2), ("B", "C", 1), ("B", "D", 5), ("C", "D", 8),
        ("C", "E", 10), ("D", "E", 2), ("D", "F", 6), ("E", "F", 2),
    ])


EXPECTED_FROM_A = {"A": 0, "B": 3, "C": 2, "D": 8, "E": 10, "F": 12}


# ---------------------------------------------------------------------- #
# Cas normaux
# ---------------------------------------------------------------------- #
def test_distances_connues(sample_graph):
    distances, _ = dijkstra(sample_graph, "A")
    assert distances == pytest.approx(EXPECTED_FROM_A)


def test_chemin_connu(sample_graph):
    result = shortest_path(sample_graph, "A", "F")
    assert result["reachable"] is True
    assert result["path"] == ["A", "C", "B", "D", "E", "F"]
    assert result["total_cost"] == 12
    assert result["hop_count"] == 5


def test_cout_du_chemin_egal_a_la_distance(sample_graph):
    for target, expected in EXPECTED_FROM_A.items():
        result = shortest_path(sample_graph, "A", target)
        assert path_cost(sample_graph, result["path"]) == pytest.approx(expected)
        assert result["total_cost"] == pytest.approx(expected)


def test_chemin_commence_a_la_source_et_finit_a_la_cible(sample_graph):
    for source in sample_graph:
        for target in sample_graph:
            path = shortest_path(sample_graph, source, target)["path"]
            assert path[0] == source and path[-1] == target


def test_symetrie_graphe_non_oriente(sample_graph):
    for u in sample_graph:
        du, _ = dijkstra(sample_graph, u)
        for v in sample_graph:
            dv, _ = dijkstra(sample_graph, v)
            assert du[v] == pytest.approx(dv[u])


def test_inegalite_triangulaire(sample_graph):
    """Pour toute arête (u, v) : d(v) <= d(u) + w(u, v). Sinon un relâchement a été oublié."""
    distances, _ = dijkstra(sample_graph, "A")
    for u, neighbors in sample_graph.items():
        for v, w in neighbors.items():
            assert distances[v] <= distances[u] + w + 1e-9


def test_graphe_oriente():
    graph = build_graph([("A", "B", 1), ("B", "C", 1), ("C", "A", 1)], directed=True)
    distances, _ = dijkstra(graph, "B")
    assert distances == {"A": 2, "B": 0, "C": 1}

```

```python

def test_format_liste_de_couples():
    graph = {"A": [("B", 2), ("C", 5)], "B": [("C", 1)], "C": []}
    assert shortest_path(graph, "A", "C")["path"] == ["A", "B", "C"]


def test_arete_multiple_garde_la_plus_courte():
    graph = build_graph([("A", "B", 9), ("A", "B", 2)])
    assert graph["A"]["B"] == 2


def test_poids_flottants_precision():
    graph = build_graph([("A", "B", 0.1), ("B", "C", 0.2)])
    result = shortest_path(graph, "A", "C")
    assert result["total_cost"] == 0.3        # arrondi à 2 décimales


def test_arret_anticipe_donne_la_meme_distance(sample_graph):
    full, _ = dijkstra(sample_graph, "A")
    early, _ = dijkstra(sample_graph, "A", target="D")
    assert early["D"] == full["D"]


def test_format_duration():
    assert format_duration(17.5) == "17 min 30 sec"
    assert format_duration(0) == "0 min 0 sec"
    assert format_duration(math.inf) == "inaccessible"


# ---------------------------------------------------------------------- #
# Cas limites
# ---------------------------------------------------------------------- #
def test_graphe_non_connexe():
    graph = build_graph([("A", "B", 1), ("C", "D", 1)])
    distances, predecessors = dijkstra(graph, "A")
    assert distances["C"] == math.inf and distances["D"] == math.inf
    assert predecessors["C"] is None
    assert reconstruct_path(predecessors, "A", "D", distances) is None
    result = shortest_path(graph, "A", "D")
    assert result == {"reachable": False, "path": None, "total_cost": None, "hop_count": None}


def test_cycles_terminent():
    graph = build_graph([("A", "B", 1), ("B", "C", 1), ("C", "A", 1), ("C", "D", 1)])
    assert shortest_path(graph, "A", "D")["path"] == ["A", "C", "D"]


def test_boucle_sur_soi_meme():
    graph = {"A": {"A": 5, "B": 1}, "B": {}}
    distances, _ = dijkstra(graph, "A")
    assert distances == {"A": 0, "B": 1}


def test_source_egale_cible(sample_graph):
    result = shortest_path(sample_graph, "C", "C")
    assert result["path"] == ["C"]
    assert result["total_cost"] == 0
    assert result["hop_count"] == 0


def test_graphe_vide_leve_une_erreur_propre():
    with pytest.raises(ValueError):
        dijkstra({}, "A")


def test_graphe_a_un_seul_noeud():
    distances, predecessors = dijkstra({"A": {}}, "A")
    assert distances == {"A": 0}
    assert predecessors == {"A": None}


def test_noeud_isole():
    graph = build_graph([("A", "B", 1)])
    graph["Z"] = {}
    assert shortest_path(graph, "A", "Z")["reachable"] is False


def test_noeud_present_seulement_en_destination():
    graph = {"A": {"B": 3}}          # B n'a pas d'entrée propre
    distances, _ = dijkstra(graph, "A")
    assert distances["B"] == 3
```

```python


def test_source_inexistante(sample_graph):
    with pytest.raises(ValueError):
        dijkstra(sample_graph, "INCONNU")


def test_cible_inexistante(sample_graph):
    with pytest.raises(ValueError):
        shortest_path(sample_graph, "A", "INCONNU")


def test_poids_negatif_refuse():
    with pytest.raises(ValueError):
        dijkstra({"A": {"B": -1}, "B": {}}, "A")


# ---------------------------------------------------------------------- #
# T-W12 : version naïve, mêmes résultats que la version avec tas
# ---------------------------------------------------------------------- #
def test_naif_sur_le_graphe_exemple(sample_graph):
    distances, _ = dijkstra_naive(sample_graph, "A")
    assert distances == pytest.approx(EXPECTED_FROM_A)
    result = shortest_path(sample_graph, "A", "F", use_heap=False)
    assert result["path"] == ["A", "C", "B", "D", "E", "F"]


def test_tas_et_naif_identiques_sur_tout_le_site():
    """Depuis chaque noeud du site, les deux versions donnent les mêmes distances."""
    for source in SITE_GRAPH:
        d_heap, _ = dijkstra(SITE_GRAPH, source)
        d_naive, _ = dijkstra_naive(SITE_GRAPH, source)
        for node in SITE_GRAPH:
            assert d_heap[node] == pytest.approx(d_naive[node]), (source, node)


def test_naif_graphe_non_connexe():
    graph = build_graph([("A", "B", 1), ("C", "D", 1)])
    distances, _ = dijkstra_naive(graph, "A")
    assert distances["C"] == math.inf
    assert shortest_path(graph, "A", "D", use_heap=False)["reachable"] is False


def test_naif_source_inexistante(sample_graph):
    with pytest.raises(ValueError):
        dijkstra_naive(sample_graph, "INCONNU")
```

Tests du tas binaire `tests/test_min_heap.py` (81 lignes)

```python
"""Tests unitaires du tas binaire (T-W2)."""

import random

import pytest

from src.min_heap import BinaryHeap


def test_heapify_respecte_la_propriete_de_tas():
    heap = BinaryHeap.heapify([9, 4, 7, 1, 8, 2, 6, 3, 5])
    assert heap.is_valid(), f"Propriété de tas violée : {heap!r}"
    assert heap.size() == 9


def test_heapify_liste_vide():
    heap = BinaryHeap.heapify([])
    assert heap.is_empty()


def test_insert_met_a_jour_taille_et_propriete():
    heap = BinaryHeap()
    for i, value in enumerate([5, 3, 8, 1, 9, 2]):
        heap.insert(value)
        assert heap.size() == i + 1
        assert heap.is_valid(), f"Tas invalide après insert({value}) : {heap!r}"


def test_extract_min_renvoie_les_elements_dans_l_ordre():
```

```python
    values = list(range(100))
    random.Random(42).shuffle(values)
    heap = BinaryHeap()
    for v in values:
        heap.insert(v)
    extracted = [heap.extract_min() for _ in range(len(values))]
    assert extracted == sorted(values)
    assert heap.is_empty()


def test_get_min_ne_modifie_pas_le_tas():
    heap = BinaryHeap.heapify([4, 2, 6])
    before = heap.to_list()
    assert heap.get_min() == 2
    assert heap.get_min() == 2
    assert heap.to_list() == before
    assert heap.size() == 3


def test_redimensionnement_conserve_les_elements():
    heap = BinaryHeap(capacity=2)
    for v in [10, 4, 7, 1, 3]:
        heap.insert(v)
    assert heap.capacity >= 5, "Le tableau interne aurait dû être agrandi"
    assert sorted(heap.to_list()) == [1, 3, 4, 7, 10]
    assert [heap.extract_min() for _ in range(5)] == [1, 3, 4, 7, 10]


def test_doublons_et_tuples():
    heap = BinaryHeap()
    for item in [(2.0, 0, "B"), (1.0, 1, "A"), (2.0, 2, "C"), (1.0, 3, "D")]:
        heap.insert(item)
    assert [heap.extract_min()[2] for _ in range(4)] == ["A", "D", "B", "C"]


def test_operations_sur_tas_vide_levent_une_erreur():
    heap = BinaryHeap()
    with pytest.raises(IndexError):
        heap.extract_min()
    with pytest.raises(IndexError):
        heap.get_min()


def test_capacite_invalide():
    with pytest.raises(ValueError):
        BinaryHeap(capacity=0)


def test_str_et_repr():
    heap = BinaryHeap.heapify([3, 1])
    assert str(heap) == "BinaryHeap([1, 3])"
    assert "size=2" in repr(heap)
```

Tests du service de routage `tests/test_routing_service.py` (66 lignes)

```python
"""Tests de la logique métier de l'endpoint (T-W5, T-W7), sans serveur HTTP."""

import json

import pytest

from src.routing_service import (
    InvalidIdentifierError,
    UnknownResourceError,
    compute_emergency_path,
)

EXPECTED_KEYS = {
    "status", "message", "coordinator_id", "target_hive_id", "start_node", "total_cost",
    "total_cost_formatted", "path", "hop_count", "calculation_time_ms", "algorithm_version",
}


def test_itineraire_connu():
    result = compute_emergency_path("C01", "H07")
    assert result["status"] == "success"
    assert result["path"] == ["BASE_NORD", "J1", "J2", "J3", "J4", "H07"]
    assert result["total_cost"] == 17.5
```

```
    assert result["total_cost_formatted"] == "17 min 30 sec"
    assert result["hop_count"] == 5


def test_structure_json_complete_et_serialisable():
    result = compute_emergency_path("C02", "H04")
    assert set(result) == EXPECTED_KEYS
    json.dumps(result)                              # ne doit pas lever d'exception
    assert result["calculation_time_ms"] > 0


def test_ids_insensibles_a_la_casse_et_aux_espaces():
    assert compute_emergency_path(" c01 ", "h07")["total_cost"] == 17.5


def test_ruche_inaccessible_renvoie_les_valeurs_par_defaut():
    result = compute_emergency_path("C01", "H13")
    assert result["status"] == "error"
    assert result["path"] == []
    assert result["total_cost"] is None
    assert result["hop_count"] == 0
    assert result["message"]


@pytest.mark.parametrize("coordinator, hive", [("X01", "H01"), ("C01", "R01"), ("C1", "H01")])
def test_format_invalide(coordinator, hive):
    with pytest.raises(InvalidIdentifierError):
        compute_emergency_path(coordinator, hive)


def test_type_invalide():
    with pytest.raises(InvalidIdentifierError):
        compute_emergency_path(1, "H01")


def test_ruche_inexistante():
    with pytest.raises(UnknownResourceError):
        compute_emergency_path("C01", "H99")


def test_coordinateur_inexistant():
    with pytest.raises(UnknownResourceError):
        compute_emergency_path("C09", "H01")
```

Tests de l'API `tests/test_api.py` (84 lignes)

```
"""Tests HTTP de l'endpoint POST /api/v1/routing/emergency-path (T-W5, T-W6, T-W7).

Nécessite fastapi et httpx (voir requirements.txt).
"""

from fastapi.testclient import TestClient

from src.api import app

client = TestClient(app)
URL = "/api/v1/routing/emergency-path"


def test_post_valide_renvoie_200_et_le_chemin():
    response = client.post(URL, json={"coordinator_id": "C01", "target_hive_id": "H07"})
    assert response.status_code == 200
    assert response.headers["content-type"].startswith("application/json")
    body = response.json()
    assert body["status"] == "success"
    assert body["total_cost"] == 17.5
    assert body["path"] == ["BASE_NORD", "J1", "J2", "J3", "J4", "H07"]
    assert body["calculation_time_ms"] >= 0


def test_champs_optionnels_acceptes():
    response = client.post(URL, json={
        "coordinator_id": "C02", "target_hive_id": "H10",
        "urgence_niveau": "critique", "metadata": {"alerte": "anomalie_thermique"},
    })
    assert response.status_code == 200
```

```python
def test_ids_en_minuscules_acceptes():
    response = client.post(URL, json={"coordinator_id": "c01", "target_hive_id": "h07"})
    assert response.status_code == 200


def test_champ_manquant_renvoie_422():
    response = client.post(URL, json={"coordinator_id": "C01"})
    assert response.status_code == 422


def test_mauvais_type_renvoie_422():
    response = client.post(URL, json={"coordinator_id": 1, "target_hive_id": "H07"})
    assert response.status_code == 422


def test_mauvais_format_renvoie_422():
    response = client.post(URL, json={"coordinator_id": "C01", "target_hive_id": "RUCHE7"})
    assert response.status_code == 422


def test_niveau_urgence_inconnu_renvoie_422():
    response = client.post(URL, json={"coordinator_id": "C01", "target_hive_id": "H07",
                                      "urgence_niveau": "apocalypse"})
    assert response.status_code == 422


def test_ruche_inexistante_renvoie_404():
    response = client.post(URL, json={"coordinator_id": "C01", "target_hive_id": "H99"})
    assert response.status_code == 404


def test_coordinateur_inexistant_renvoie_404():
    response = client.post(URL, json={"coordinator_id": "C09", "target_hive_id": "H01"})
    assert response.status_code == 404


def test_ruche_inaccessible_renvoie_status_error():
    response = client.post(URL, json={"coordinator_id": "C01", "target_hive_id": "H13"})
    assert response.status_code == 200
    body = response.json()
    assert body["status"] == "error"
    assert body["path"] == [] and body["total_cost"] is None


def test_get_renvoie_405():
    assert client.get(URL).status_code == 405


def test_documentation_openapi_disponible():
    assert client.get("/docs").status_code == 200
    schema = client.get("/openapi.json").json()
    assert URL in schema["paths"]
```

4.2 Tests E2E (Playwright)

Un vrai serveur uvicorn est lancé et un navigateur Chromium pilote Swagger UI. Résultat : **8 passed in 5.35s**. Chaque test vérifie le code HTTP et le contenu de la réponse, puis enregistre une capture nommée.

```
============================= test session starts ==============================
collecting ... collected 8 items
tests/e2e/test_e2e_playwright.py::test_01_swagger_ui_documente_l_endpoint PASSED [ 12%]
tests/e2e/test_e2e_playwright.py::test_02_swagger_chemin_nominal_200 PASSED [ 25%]
tests/e2e/test_e2e_playwright.py::test_03_swagger_ruche_inexistante_404 PASSED [ 37%]
tests/e2e/test_e2e_playwright.py::test_04_swagger_identifiant_invalide_422 PASSED [ 50%]
tests/e2e/test_e2e_playwright.py::test_05_swagger_ruche_isolee_statut_error PASSED [ 62%]
tests/e2e/test_e2e_playwright.py::test_06_schemas_openapi_visibles PASSED [ 75%]
tests/e2e/test_e2e_playwright.py::test_07_api_http_cas_nominal_et_methodes PASSED [ 87%]
tests/e2e/test_e2e_playwright.py::test_08_openapi_json_et_capture_reponse_brute PASSED [100%]
============================== 8 passed in 5.35s ===============================
```

Code des tests E2E `tests/e2e/test_e2e_playwright.py` (161 lignes)

```python
"""Tests E2E (Playwright) de l'API QueenBee via Swagger UI et le client HTTP.

Lance un vrai serveur uvicorn, pilote un vrai navigateur Chromium, vérifie les
réponses et enregistre des captures d'écran nommées dans docs/captures/.

    python -m pytest tests/e2e -v
"""

import json
import socket
import subprocess
import sys
import time
from pathlib import Path

import pytest

playwright_sync = pytest.importorskip("playwright.sync_api")
sync_playwright = playwright_sync.sync_playwright

ROOT = Path(__file__).resolve().parents[2]
CAPTURES = ROOT / "docs" / "captures"
ENDPOINT = "/api/v1/routing/emergency-path"


def _free_port():
    with socket.socket() as s:
        s.bind(("127.0.0.1", 0))
        return s.getsockname()[1]


@pytest.fixture(scope="module")
def base_url():
    port = _free_port()
    proc = subprocess.Popen(
        [sys.executable, "-m", "uvicorn", "src.api:app", "--port", str(port), "--log-level", "warning"],
        cwd=ROOT)
    url = f"http://127.0.0.1:{port}"
    for _ in range(60):
        try:
            socket.create_connection(("127.0.0.1", port), timeout=0.5).close()
            break
        except OSError:
            time.sleep(0.25)
    yield url
    proc.terminate()
    proc.wait(timeout=10)


@pytest.fixture(scope="module")
def browser_page(base_url):
    CAPTURES.mkdir(parents=True, exist_ok=True)
    with sync_playwright() as pw:
        browser = pw.chromium.launch()
        page = browser.new_page(viewport={"width": 1280, "height": 900})
        import os
        def _cdn(route):
            name = route.request.url.split("/")[-1].split("?")[0]
            p = "/tmp/sw/node_modules/swagger-ui-dist/" + name
            route.fulfill(path=p) if os.path.exists(p) else route.abort()
        page.route("**/*", lambda r: _cdn(r) if "jsdelivr" in r.request.url or "tiangolo" in r.request.url
else r.continue_())
        page.goto(f"{base_url}/docs")
        page.wait_for_selector(".opblock-post .opblock-summary", timeout=30000)
        yield page
        browser.close()


def _shot(page, name):
    page.screenshot(path=str(CAPTURES / name))


def _execute_in_swagger(page, body):
    """Ouvre l'opération POST, remplace le corps JSON et clique sur Execute."""
    block = page.locator(".opblock-post")
    if "is-open" not in (block.get_attribute("class") or ""):
        page.locator(".opblock-post .opblock-summary").click()
    page.wait_for_selector("button.try-out__btn")
    if "cancel" not in (page.locator("button.try-out__btn").first.get_attribute("class") or ""):
        page.locator("button.try-out__btn").first.click()
```

```
    page.wait_for_selector(".body-param__text")
    page.locator(".body-param__text").fill(json.dumps(body, indent=2))
    page.locator("button.execute").click()
    page.wait_for_selector(".live-responses-table tbody .response-col_status", timeout=10000)
    return page.locator(".live-responses-table tbody .response-col_status").first.inner_text().strip()


def test_01_swagger_ui_documente_l_endpoint(browser_page):
    page = browser_page
    assert page.locator(".opblock-post .opblock-summary").count() == 1
    assert ENDPOINT in page.locator(".opblock-summary-path").inner_text()
    _shot(page, "01_swagger_documentation_endpoint.png")


def test_02_swagger_chemin_nominal_200(browser_page):
    page = browser_page
    status = _execute_in_swagger(page, {"coordinator_id": "C01", "target_hive_id": "H07",
                                        "urgence_niveau": "critique"})
    assert status.startswith("200")
    body = page.locator(".live-responses-table .microlight").first.inner_text()
    data = json.loads(body)
    assert data["status"] == "success" and data["path"][0] == "BASE_NORD" and data["path"][-1] == "H07"
    assert data["total_cost"] == 17.5
    page.locator(".live-responses-table").scroll_into_view_if_needed()
    _shot(page, "02_swagger_requete_nominale_200_succes.png")


def test_03_swagger_ruche_inexistante_404(browser_page):
    page = browser_page
    status = _execute_in_swagger(page, {"coordinator_id": "C01", "target_hive_id": "H99"})
    assert status.startswith("404")
    page.locator(".live-responses-table").scroll_into_view_if_needed()
    _shot(page, "03_swagger_ruche_inexistante_404.png")


def test_04_swagger_identifiant_invalide_422(browser_page):
    page = browser_page
    status = _execute_in_swagger(page, {"coordinator_id": "ZZZ", "target_hive_id": "H07"})
    assert status.startswith("422")
    page.locator(".live-responses-table").scroll_into_view_if_needed()
    _shot(page, "04_swagger_identifiant_invalide_422.png")


def test_05_swagger_ruche_isolee_statut_error(browser_page):
    page = browser_page
    status = _execute_in_swagger(page, {"coordinator_id": "C02", "target_hive_id": "H13"})
    assert status.startswith("200")
    data = json.loads(page.locator(".live-responses-table .microlight").first.inner_text())
    assert data["status"] == "error" and data["path"] == [] and data["total_cost"] is None
    page.locator(".live-responses-table").scroll_into_view_if_needed()
    _shot(page, "05_swagger_ruche_isolee_aucun_itineraire.png")

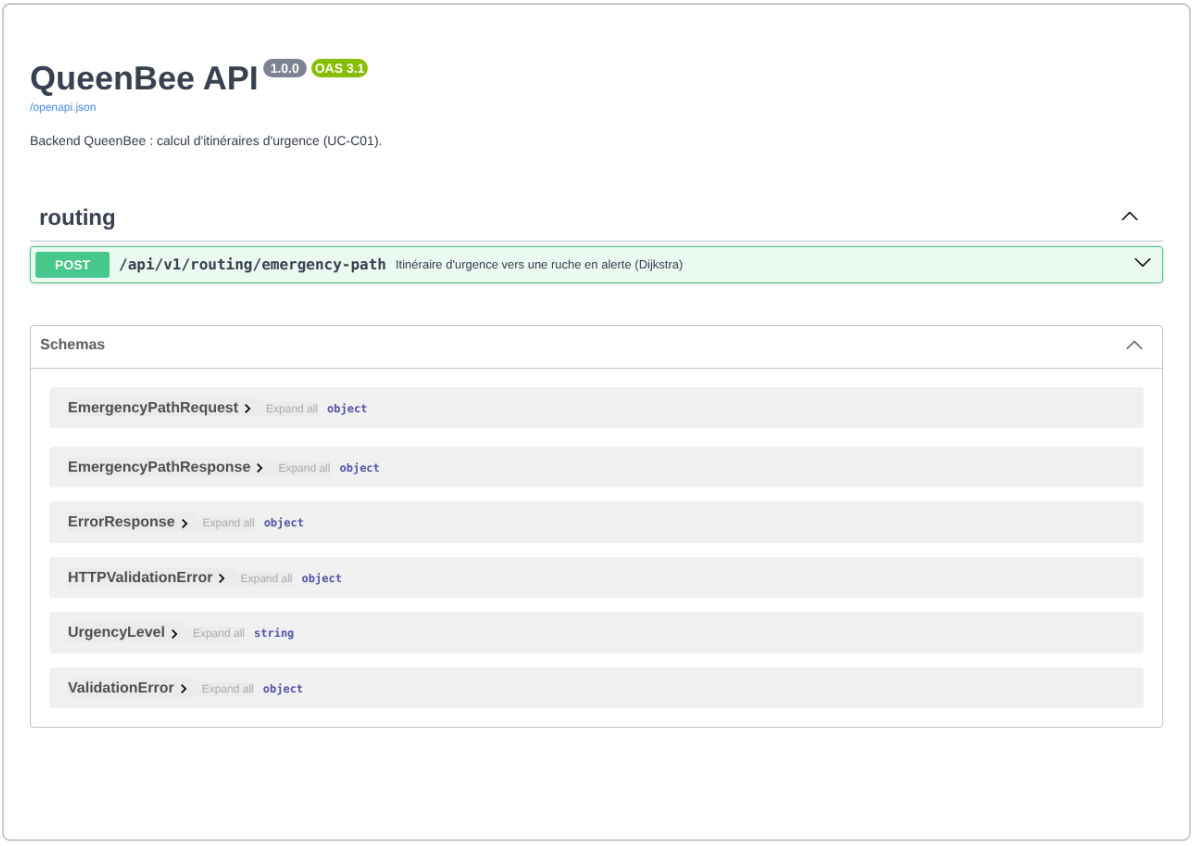

def test_06_schemas_openapi_visibles(browser_page):
    page = browser_page
    page.locator("section.models").scroll_into_view_if_needed()
    if "is-open" not in (page.locator("section.models").get_attribute("class") or ""):
        page.locator("section.models h4 button").click()
    page.get_by_text("EmergencyPathRequest").first.wait_for(timeout=5000)
    _shot(page, "06_swagger_schemas_pydantic.png")


def test_07_api_http_cas_nominal_et_methodes(base_url, browser_page):
    ctx = browser_page.context.request
    ok = ctx.post(base_url + ENDPOINT, data={"coordinator_id": "C02", "target_hive_id": "H12"})
    assert ok.status == 200
    data = ok.json()
    assert set(["total_cost", "path", "calculation_time_ms"]) <= set(data)
    assert data["calculation_time_ms"] >= 0 and data["path"][-1] == "H12"
    assert ctx.get(base_url + ENDPOINT).status == 405
    assert ctx.post(base_url + ENDPOINT, data={"coordinator_id": "C01"}).status == 422
    assert ctx.post(base_url + ENDPOINT, data={"coordinator_id": "C09", "target_hive_id": "H01"}).status
== 404

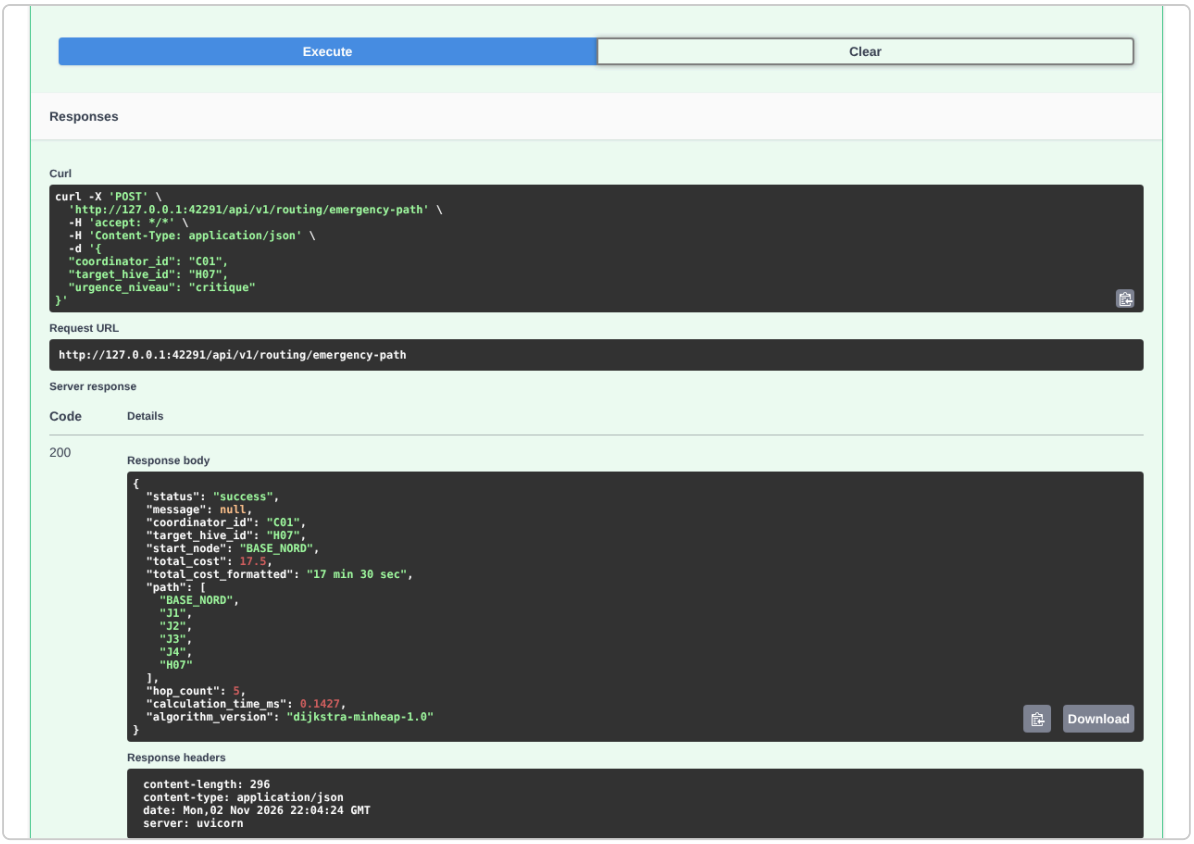

def test_08_openapi_json_et_capture_reponse_brute(base_url, browser_page):
    page = browser_page.context.browser.new_page(viewport={'width': 1000, 'height': 700})
    page.goto(base_url + "/openapi.json")
    spec = json.loads(page.inner_text("body"))
    assert ENDPOINT in spec["paths"] and "post" in spec["paths"][ENDPOINT]
```

```
159     _shot(page, "07_openapi_json_contrat_api.png")
160     page.close()
```

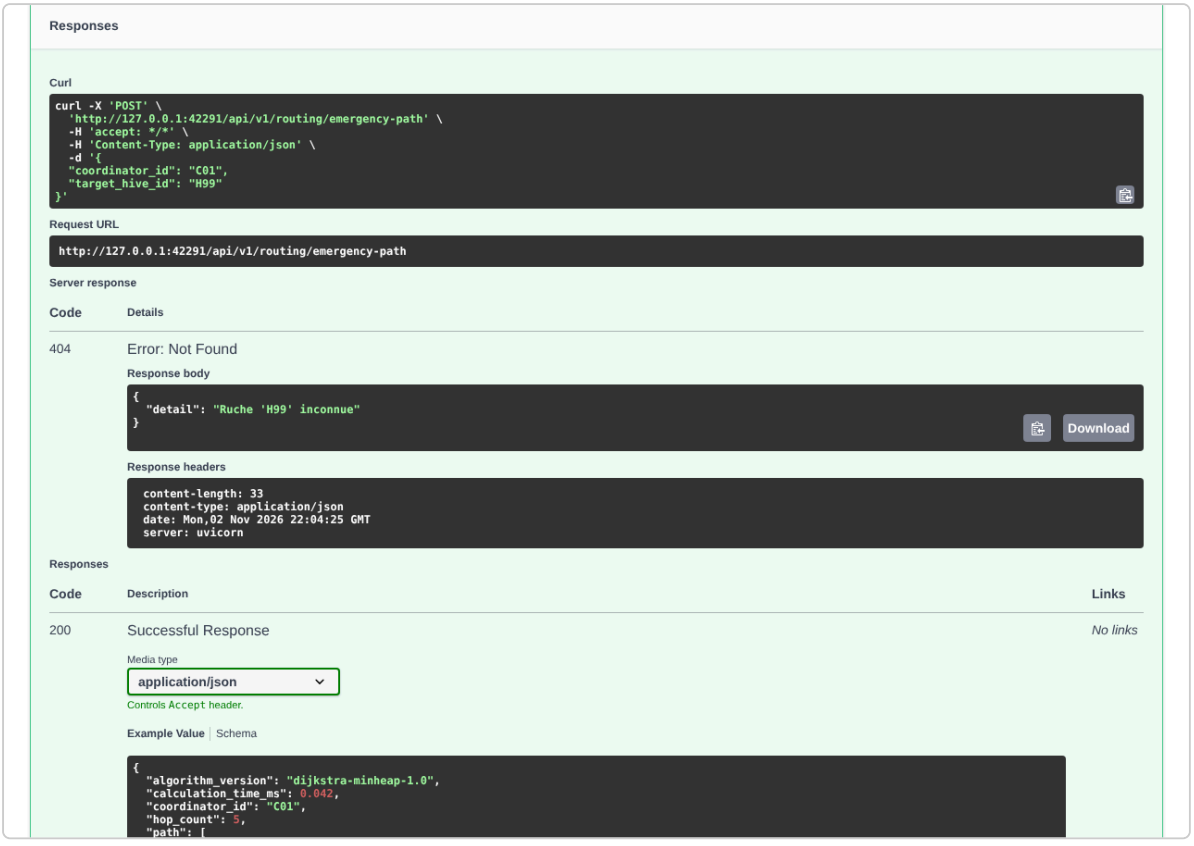
4.3 Captures d'écran des tests E2E



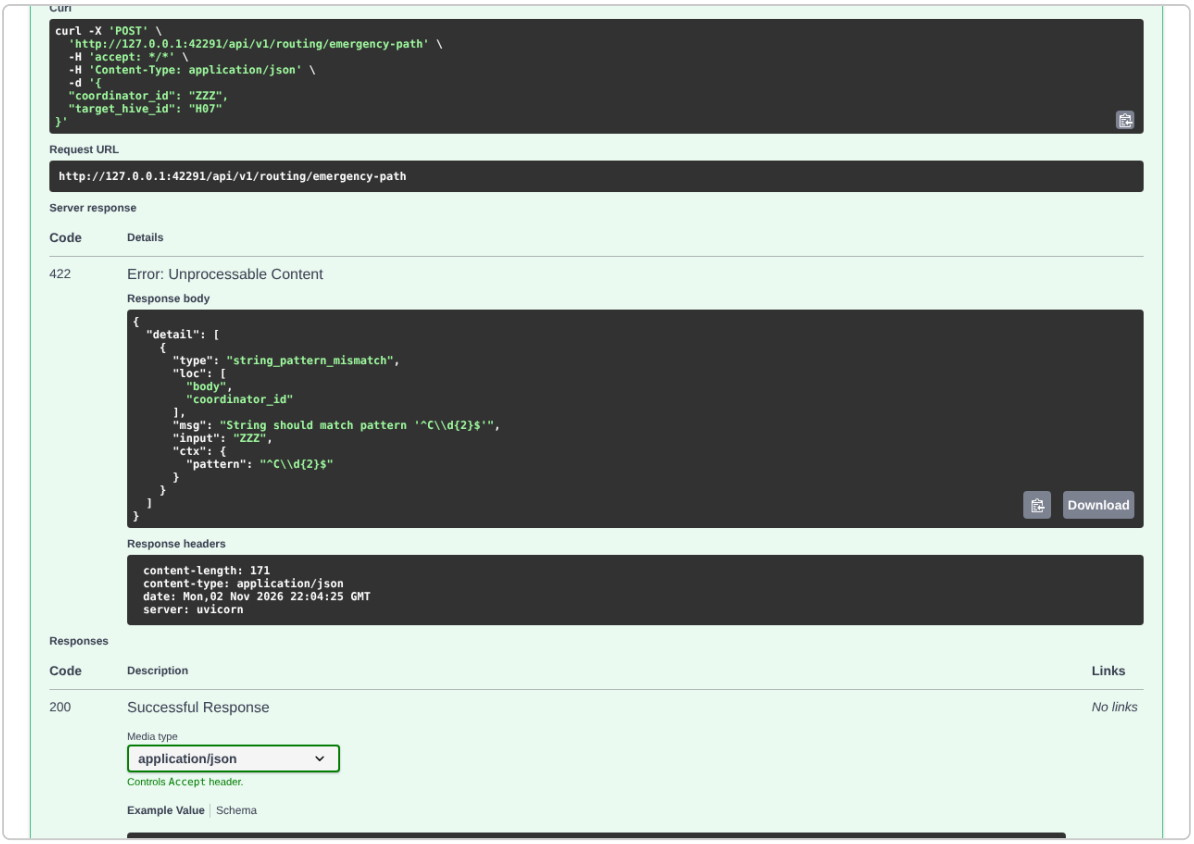
01_swagger_documentation_endpoint.png : Documentation OpenAPI : endpoint POST /api/v1/routing/emergency-path exposé dans Swagger UI.



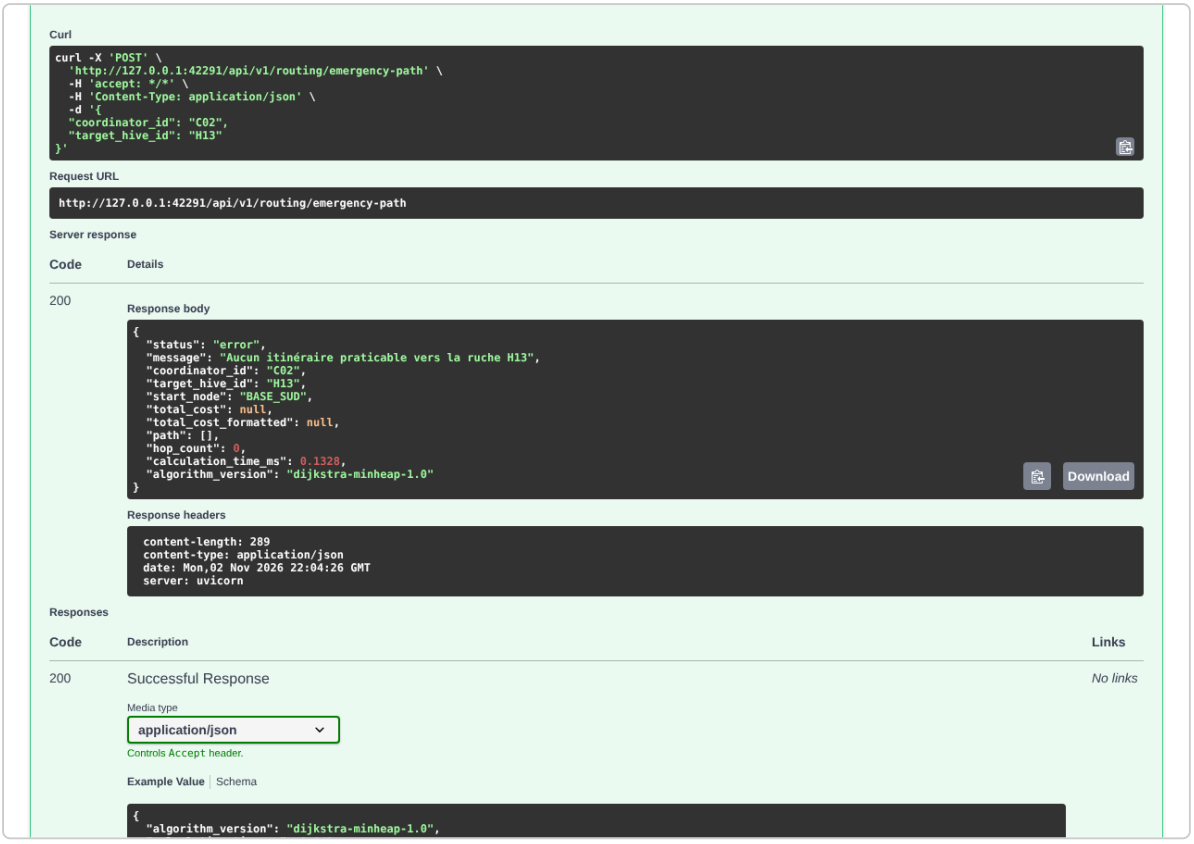
02_swagger_requete_nominale_200_succes.png : Requête nominale C01 vers H07 : HTTP 200, coût 17,5 min, chemin BASE_NORD, J1, J2, J3, J4, H07, temps de calcul en ms.



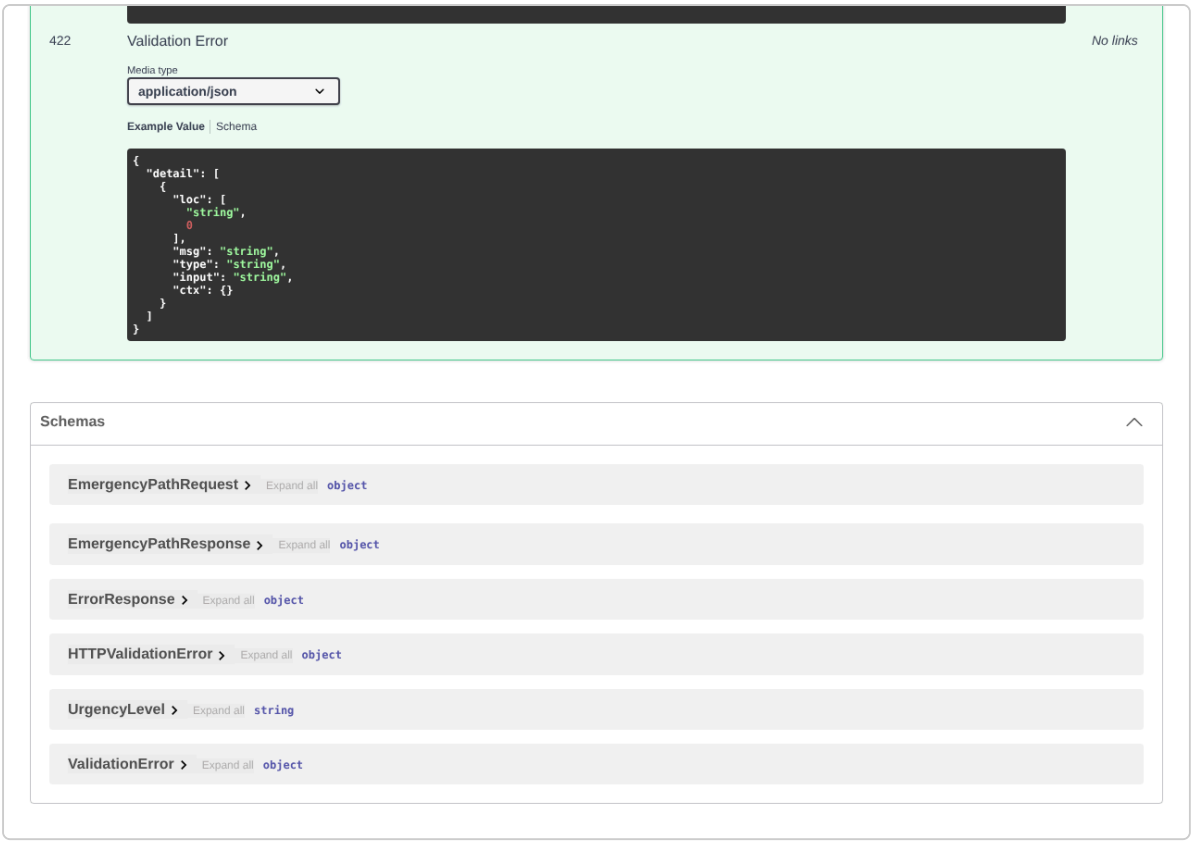
03_swagger_ruche_inexistante_404.png : Ruche inexistante H99 : HTTP 404.



04_swagger_identifiant_invalide_422.png : Identifiant mal formé « ZZZ » : validation Pydantic, HTTP 422.



05_swagger_ruche_isolee_aucun_itineraire.png : Ruche isolée H13 (graphe non connexe) : HTTP 200, status « error », chemin vide, coût null.



06_swagger_schemas_pydantic.png : Schémas Pydantic d'entrée et de sortie documentés.

Pretty-print


```
{"openapi":"3.1.0","info":{"title":"QueenBee API","description":"Backend QueenBee : calcul d'itinéraires d'urgence (UC-
C01).","version":"1.0.0"},"paths":{"/api/v1/routing/emergency-path":{"post":{"tags":["routing"],"summary":"Itinéraire d'urgence
vers une ruche en alerte (Dijkstra)","description":"Calcule le plus court chemin entre la base du coordinateur et la ruche
cible.\n\n- **200** : calcul effectué (`status` = `success`, ou `error` si la ruche est inaccessible)\n- **400** : identifiant
invalide\n- **404** : coordinateur ou ruche inconnu\n- **422** : corps de requête invalide (champ manquant, mauvais type,
mauvais format)","operationId":"emergency_path_api_v1_routing_emergency_path_post","requestBody":{"content":
{"application/json":{"schema":{"$ref":"#/components/schemas/EmergencyPathRequest"}}},"required":true},"responses":{"200":
{"description":"Successful Response","content":{"application/json":{"schema":
{"$ref":"#/components/schemas/EmergencyPathResponse"}}}},"400":{"description":"Bad Request","content":{"application/json":
{"schema":{"$ref":"#/components/schemas/ErrorResponse"}}}},"404":{"description":"Not Found","content":{"application/json":
{"schema":{"$ref":"#/components/schemas/ErrorResponse"}}}},"422":{"description":"Validation Error","content":
{"application/json":{"schema":{"$ref":"#/components/schemas/HTTPValidationError"}}}}}}},"components":{"schemas":
{"EmergencyPathRequest":{"properties":{"coordinator_id":{"type":"string","pattern":"^C\\d{2}$","title":"Coordinator
Id","description":"Identifiant du coordinateur régional (point de départ), format CXX.","examples":["C01"]},"target_hive_id":
{"type":"string","pattern":"^H\\d{2}$","title":"Target Hive Id","description":"Identifiant de la ruche en alerte critique,
format HXX.","examples":["H07"]},"urgence_niveau":{"anyOf":[{"$ref":"#/components/schemas/UrgencyLevel"},
{"type":"null"}],"description":"Niveau d'urgence de l'alerte (optionnel)."},"metadata":{"anyOf":
[{"additionalProperties":true,"type":"object"},{"type":"null"}],"title":"Metadata","description":"Informations libres sur
l'alerte (optionnel).","examples":[{"alerte":"chute_de_poids"}]}},"type":"object","required":
["coordinator_id","target_hive_id"],"title":"EmergencyPathRequest","examples":
[{"coordinator_id":"C01","target_hive_id":"H07","urgence_niveau":"critique"}]},"EmergencyPathResponse":{"properties":{"status":
{"type":"string","title":"Status","description":"\"success\" ou \"error\" (ruche inaccessible)."},"message":{"anyOf":
[{"type":"string"},{"type":"null"}],"title":"Message","description":"Détail en cas d'erreur."},"coordinator_id":
{"type":"string","title":"Coordinator Id"},"target_hive_id":{"type":"string","title":"Target Hive Id"},"start_node":
{"type":"string","title":"Start Node","description":"Noeud de départ du coordinateur dans le graphe."},"total_cost":{"anyOf":
[{"type":"number"},{"type":"null"}],"title":"Total Cost","description":"Temps de trajet total en minutes (2 décimales), null si
inaccessible."},"total_cost_formatted":{"anyOf":[{"type":"string"},{"type":"null"}],"title":"Total Cost
Formatted","description":"Ex. \"12 min 30 sec\"."},"path":{"items":
{"type":"string"},"type":"array","title":"Path","description":"Points de passage ordonnés, de la base à la
ruche."},"hop_count":{"type":"integer","title":"Hop Count","description":"Nombre de tronçons
parcourus."},"calculation_time_ms":{"type":"number","title":"Calculation Time Ms","description":"Temps de calcul de
l'algorithme (ms)."},"algorithm_version":{"type":"string","title":"Algorithm Version"}},"type":"object","required":
["status","coordinator_id","target_hive_id","start_node","path","hop_count","calculation_time_ms","algorithm_version"],"title":
"EmergencyPathResponse","examples":[{"algorithm_version":"dijkstra-minheap-
1.0","calculation_time_ms":0.042,"coordinator_id":"C01","hop_count":5,"path":
["BASE_NORD","J1","J2","J3","J4","H07"],"start_node":"BASE_NORD","status":"success","target_hive_id":"H07","total_cost":17.5,"t
otal_cost_formatted":"17 min 30 sec"}]},"ErrorResponse":{"properties":{"detail":
{"type":"string","title":"Detail"}},"type":"object","required":["detail"],"title":"ErrorResponse"},"HTTPValidationError":
{"properties":{"detail":{"items":
{"$ref":"#/components/schemas/ValidationError"},"type":"array","title":"Detail"}},"type":"object","title":"HTTPValidationError"
},"UrgencyLevel":{"type":"string","enum":["critique","haute","moyenne"],"title":"UrgencyLevel"},"ValidationError":
{"properties":{"loc":{"items":{"anyOf":[{"type":"string"},{"type":"integer"}]},"type":"array","title":"Location"},"msg":
{"type":"string","title":"Message"},"type":{"type":"string","title":"Error Type"},"input":{"title":"Input"},"ctx":
{"type":"object","title":"Context"}},"type":"object","required":["loc","msg","type"],"title":"ValidationError"}}}}
```


07_openapi_json_contrat_api.png : Contrat OpenAPI brut (/openapi.json).

5. Note d'analyse de complexité

Notations : $n = |V|$ (nombre de sommets), $m = |E|$ (nombre d'arêtes), s = sommet source, $w(u, v) \geq 0$ le poids d'une arête, $d[v]$ la distance provisoire et $\delta(s, v)$ la vraie distance minimale.

1. Correction de l'algorithme

Invariant. Au moment où un sommet u est extrait du tas pour la première fois (et marqué `visited`), on a $d[u] = \delta(s, u)$.

Justification. Supposons le contraire et prenons le premier sommet u extrait avec $d[u] > \delta(s, u)$. Sur un plus court chemin de s à u, soit y le premier sommet non encore visité et x son prédécesseur, qui est déjà visité. Quand x a été traité, l'arête (x, y) a été relâchée, donc $d[y] = \delta(s, y)$. Comme les poids sont positifs ou nuls, $\delta(s, y) \leq \delta(s, u) < d[u]$. On a donc $d[y] < d[u]$, et le tas aurait dû extraire y avant u : c'est une contradiction.

L'hypothèse $w \geq 0$ est indispensable. C'est pourquoi le code rejette les poids négatifs (`ValueError`).

2. Complexité de la version avec tas binaire : $O((|V| + |E|) \log |V|)$

L'implémentation utilise la **suppression paresseuse** : quand une distance s'améliore, on ré-insère le sommet au lieu de faire un *decrease-key*, et les entrées périmées sont ignorées à l'extraction.

Étape 1 : initialisation. La création des tableaux `distances` et `predecessors` coûte $O(n)$.

Étape 2 : nombre de relâchements. Un sommet n'est développé qu'une seule fois, grâce à l'ensemble `visited`. Ses arêtes sortantes ne sont donc parcourues qu'une fois. Le nombre total de relâchements est $\Sigma \deg(u) = O(m)$.

Étape 3 : nombre d'opérations sur le tas.

- Insertions : 1 pour la source, plus au plus 1 par relâchement réussi. On a donc au plus $m + 1$ insertions.
- Extractions : on n'extrait que ce qu'on a inséré, donc au plus $m + 1$ extractions.
- Taille maximale du tas : $m + 1$.

Étape 4 : coût d'une opération. Le tas est un arbre binaire presque complet, de hauteur $\lfloor \log_2 k \rfloor$ pour k éléments. `insert` (remontée) et `extract_min` (descente) parcourent au plus cette hauteur, soit $O(\log(m + 1))$. Comme $m \leq n^2$, on a $\log m \leq 2 \log n$, donc **$O(\log m) = O(\log n)$**.

Étape 5 : total.

$T(n, m) = O(n) + (m + 1) \cdot O(\log n)$ [insertions] $+ (m + 1) \cdot O(\log n)$ [extractions] $+ O(m)$ [relâchements] = **$O((n + m) \log n)$**

Mémoire. $O(n + m)$: les tableaux de distances et de prédécesseurs, plus un tas d'au plus $m + 1$ entrées.

Remarques.

- Avec un tas à *decrease-key* (tas indexé), on aurait n extractions et m diminutions, ce qui donne la même borne $O((n + m) \log n)$, pour un code plus complexe. La suppression paresseuse garde

la borne tout en simplifiant le tas.
- `heapify` coûte O(n) par l'algorithme de Floyd : la somme des hauteurs des nœuds est O(n). Il n'est pas utilisé dans Dijkstra, car le tas est rempli au fil de l'eau.
- Le redimensionnement du tableau du tas (doublement) coûte O(1) amorti par insertion.
- La reconstitution du chemin coûte O(longueur du chemin) ≤ O(n).
- L'arrêt anticipé, qui s'arrête dès que la ruche cible est extraite, ne change pas le pire cas mais réduit le coût moyen pour l'endpoint.

3. Complexité de la version naïve : O(|V|²)

À chacune des n itérations, le sommet non visité le plus proche est cherché par **parcours linéaire** de l'ensemble non visité, de taille n, puis n − 1, etc.

- Recherche du minimum : Σ_{k=1..n} k = n(n + 1)/2 = O(n²).
- Relâchements : O(m), chaque arête étant parcourue une fois.
- Total : O(n² + m) = **O(n²)**, puisque m ≤ n².

4. Comparaison théorique

Type de graphe	m	Avec tas	Naïf	Meilleur
Clairsemé (réseau de pistes, degré borné)	Θ(n)	O(n log n)	O(n²)	**Tas**
Dense	Θ(n²)	O(n² log n)	O(n²)	**Naïf**

La version avec tas devient avantageuse quand (n + m) log n < n², c'est-à-dire en gros quand **m < n² / log n**. Sur un graphe complet, le tas fait m · log n opérations alors que le naïf n'en fait que n².

5. Résultats du benchmark

Le protocole est implémenté dans `python -m src.benchmark` :

- graphes aléatoires connexes, poids dans [1, 20] ;
- 3 graines par taille ;
- pour chaque graphe, meilleur temps sur 3 exécutions, mesuré avec `time.perf_counter()` ;
- résultats présentés en moyenne ± écart-type sur les 3 graphes.

Seul l'algorithme est chronométré, pas la construction du graphe. À chaque mesure, on vérifie que les deux versions donnent exactement les mêmes distances. La colonne « opérations » compte les comparaisons (dans le tas, ou lors de la recherche linéaire) plus les relâchements. Environnement : Python 3.14, Windows, une seule machine.

Graphes clairsemés (m ≈ 3n, cas réaliste d'un maillage de pistes)

Nœuds	Arêtes	Tas (ms)	Naïf (ms)	Rapport naïf/tas	Opérations tas	Opérations naïf
20	60	0.149 ± 0.035	0.059 ± 0.004	0.40	230	270
50	150	0.438 ± 0.004	0.229 ± 0.001	0.52	832	1 425
100	300	0.953 ± 0.028	0.734 ± 0.002	0.77	1 881	5 350
200	600	1.759 ± 0.164	2.067 ± 0.118	1.17	4 327	20 700
500	1500	5.656 ± 0.841	13.973 ± 0.568	2.47	12 740	126 750
1000	3000	14.807 ± 0.958	74.869 ± 35.457	5.06	28 577	503 500

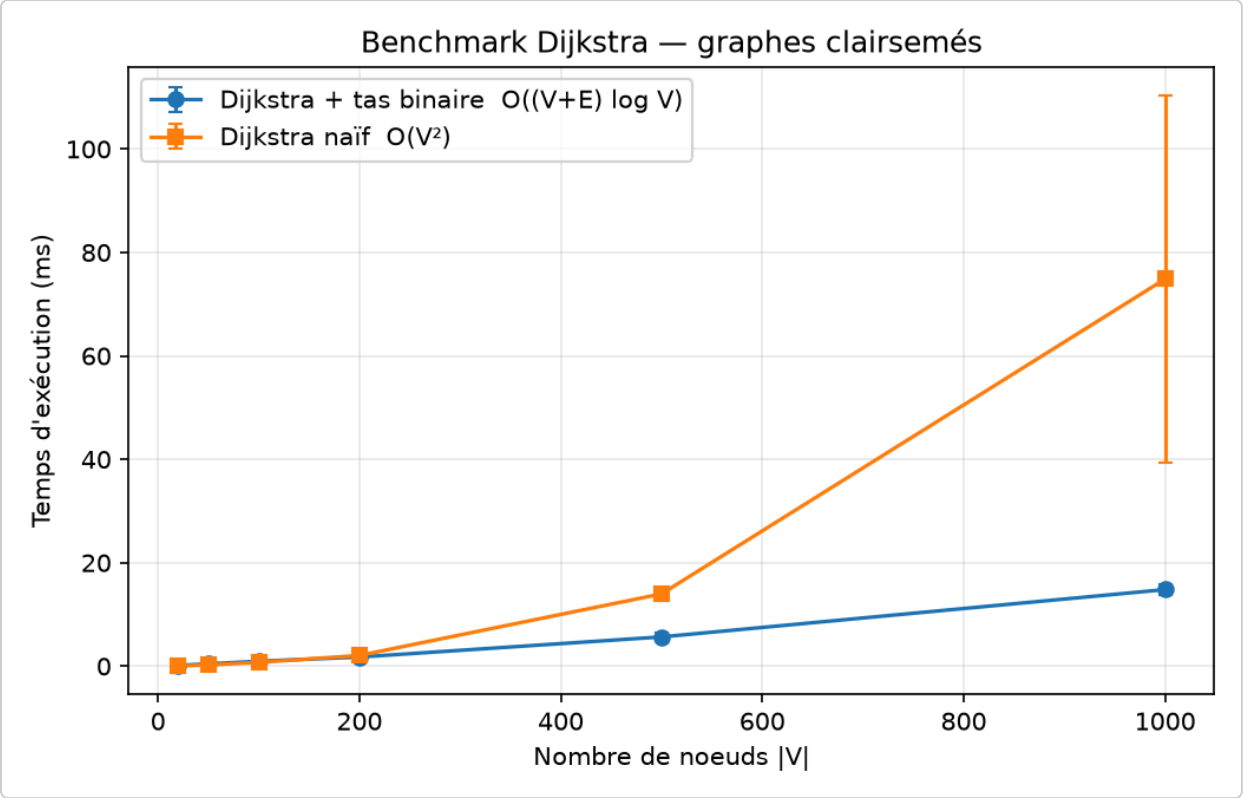


Figure : Benchmark graphes clairsemés

Graphes denses (m ≈ n²/4)

Nœuds	Arêtes	Tas (ms)	Naïf (ms)	Rapport naïf/tas	Opérations tas	Opérations naïf
20	107	0.180 ± 0.020	0.071 ± 0.001	0.39	347	317
50	641	0.937 ± 0.063	0.382 ± 0.024	0.41	2 021	1 916
100	2549	2.374 ± 0.320	1.284 ± 0.222	0.54	6 376	7 599
200	10133	7.333 ± 1.322	4.632 ± 0.534	0.63	20 239	30 233
500	62678	39.595 ± 3.468	37.408 ± 1.570	0.94	97 798	187 928

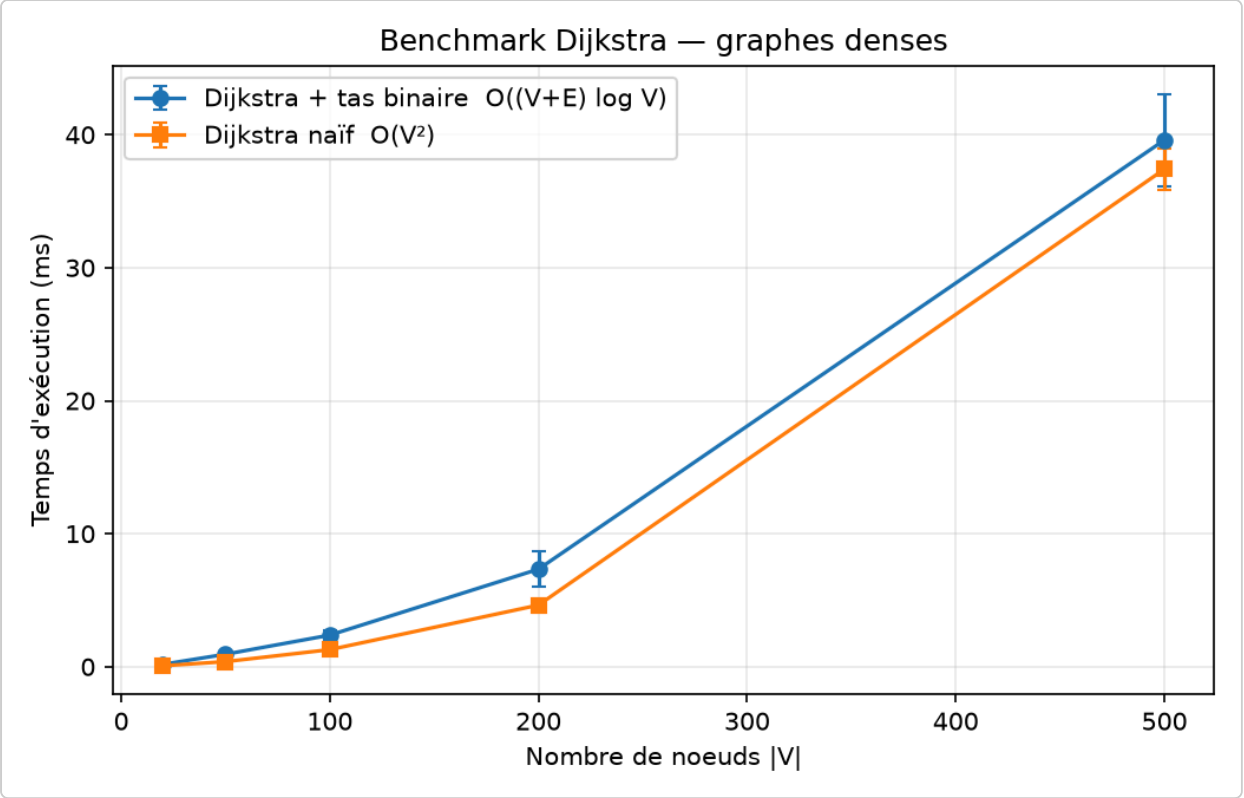


Figure : Benchmark graphes denses

6. Interprétation

Les taux de croissance confirment la théorie. Sur les graphes clairsemés, quand on passe de 200 à 500 nœuds (n multiplié par 2,5) :

- la version naïve est multipliée par **6,8**, ce qui est cohérent avec n^2, dont le facteur théorique est $2{,}5^2 = 6{,}25$;
- la version avec tas est multipliée par **3,2**, ce qui est cohérent avec n log n, dont le facteur théorique est $2{,}5 \times \log 500 / \log 200 \approx 2{,}9$.

De 500 à 1000 nœuds, on retrouve la même tendance : le naïf est multiplié par 5,4 (théorie : 4) et le tas par 2,6 (théorie : 2,2). L'écart avec la théorie y est plus grand pour le naïf, car la mesure à n = 1000 est bruitée (écart-type de 35 ms, voir plus bas).

Le nombre d'opérations, qui ne dépend pas de la machine, est plus fiable que le temps. Celui du naïf suit presque exactement $n^2/2$: 503 500 opérations pour n = 1000. À cette taille, le naïf fait **17,6 fois plus d'opérations** que le tas, pour un temps seulement 5 fois plus long. Une opération sur le tas coûte donc environ 3,5 fois plus cher qu'une comparaison de la boucle naïve.

Le seuil de rentabilité se situe entre 100 et 200 nœuds sur graphe clairsemé. Le rapport naïf/tas passe de 0,77 à 100 nœuds à 1,17 à 200 nœuds. En dessous de ce seuil, le naïf est plus rapide malgré sa complexité. Une opération sur le tas (comparaison de tuples, échanges, appels de méthodes en Python pur) coûte beaucoup plus cher qu'une comparaison dans la boucle linéaire. Les constantes cachées par la notation O dominent donc pour les petits n. Au-delà du seuil, l'écart grandit avec la taille : 2,5 fois plus rapide à 500 nœuds, 5 fois à 1000 nœuds.

Sur graphe dense, le tas ne gagne jamais. Le rapport naïf/tas reste inférieur à 1 pour toutes les tailles, et les deux versions sont presque à égalité à n = 500 (0,94). Le tas fait pourtant moins

d'opérations comptées que le naïf à partir de 100 nœuds, mais chacune coûte plus cher. C'est le résultat prévu par la section 4 : avec $m = \Theta(n^2)$, le tas est en $O(n^2 \log n)$ contre $O(n^2)$ pour le naïf.

Écarts-types. Les écarts-types sont faibles dans l'ensemble, sauf pour le naïf à n = 1000 sur graphe clairsemé (74,9 ± 35,5 ms). Cette dispersion vient de l'environnement d'exécution (autres processus, ordonnanceur du système), pas de l'algorithme : son nombre d'opérations est identique d'une exécution à l'autre. C'est pour limiter ce bruit qu'on garde le meilleur temps sur 3 répétitions. Même avec la valeur basse de l'intervalle (environ 39 ms), le naïf reste plus de 2,5 fois plus lent que le tas.

7. Conclusion pour QueenBee

Le maillage des pistes et accès d'un site est un **graphe clairsemé**. Chaque carrefour ou ruche n'est relié qu'à quelques voisins, donc $m = O(n)$. C'est exactement le cas où la version avec tas binaire est asymptotiquement et pratiquement meilleure. Le choix du tas est donc justifié pour l'endpoint `/api/v1/routing/emergency-path`, d'autant que l'arrêt anticipé sur la ruche cible réduit encore le travail.

Sur le petit graphe de démonstration (≈ 25 nœuds), le calcul prend moins de 0,1 ms quelle que soit la version. Le gain devient significatif à l'échelle du réseau complet des 400 ruches et de leurs points de passage.

Limites de l'étude.

- Les mesures sont faites en Python interprété ; dans un langage compilé, les constantes et donc le seuil de rentabilité changent.
- Les graphes sont aléatoires et non issus des données réelles du site.
- Les mesures proviennent d'une seule machine.